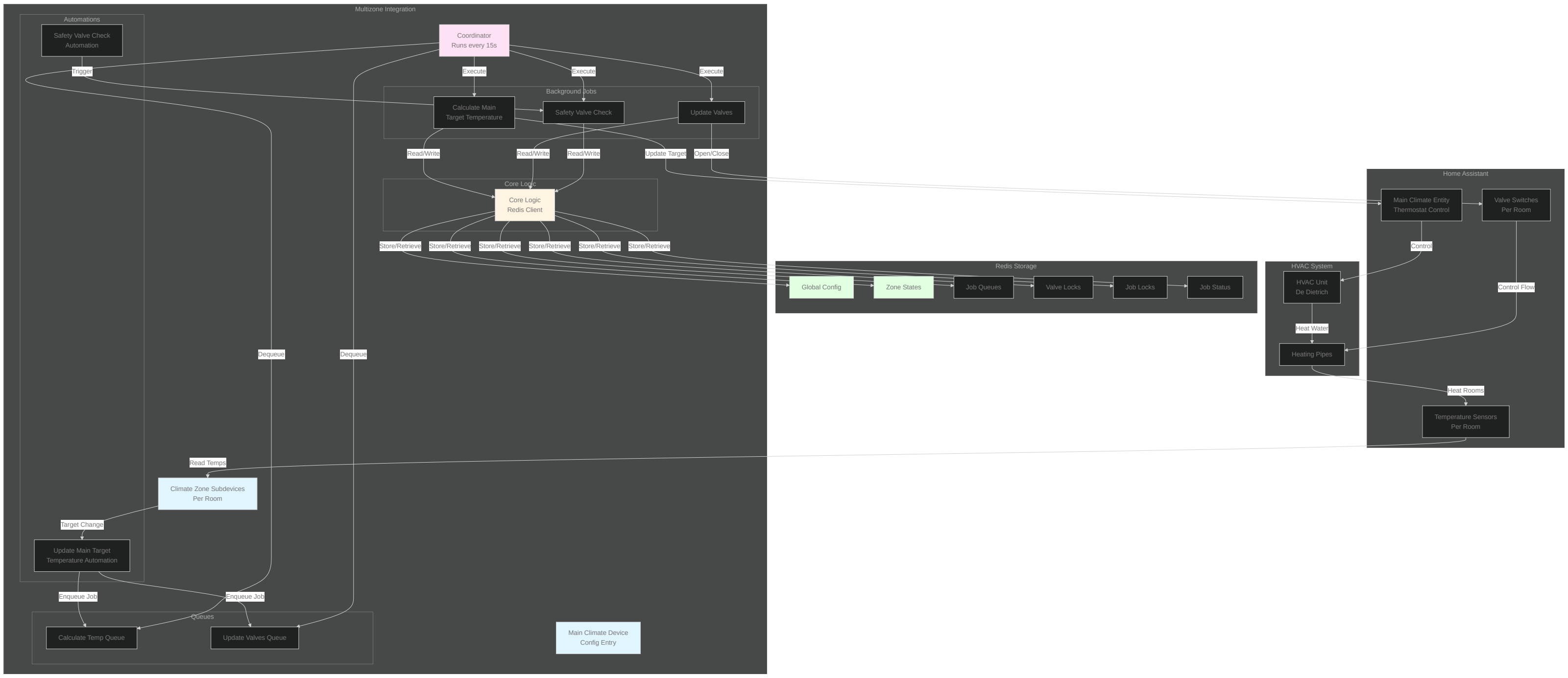
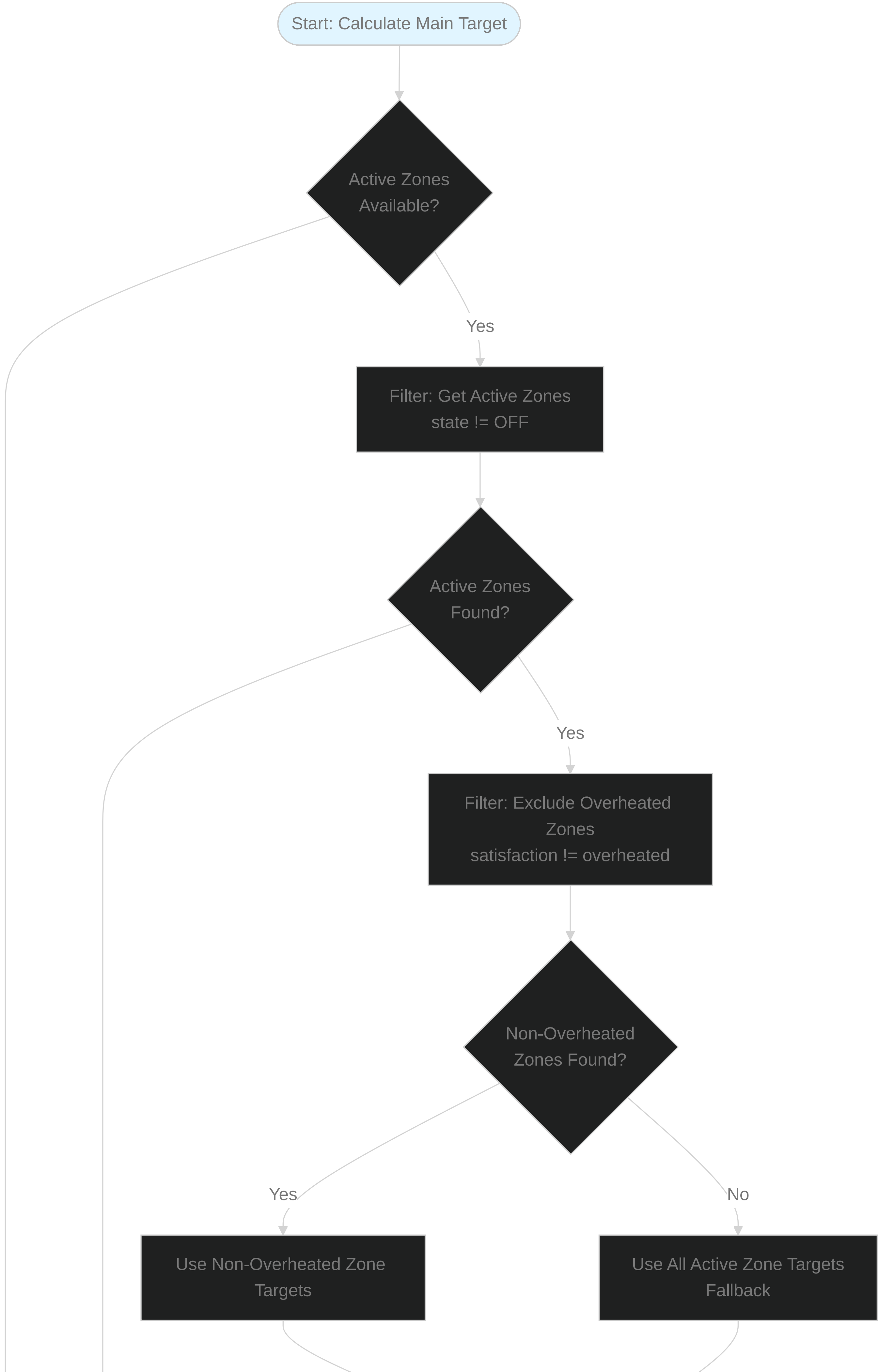
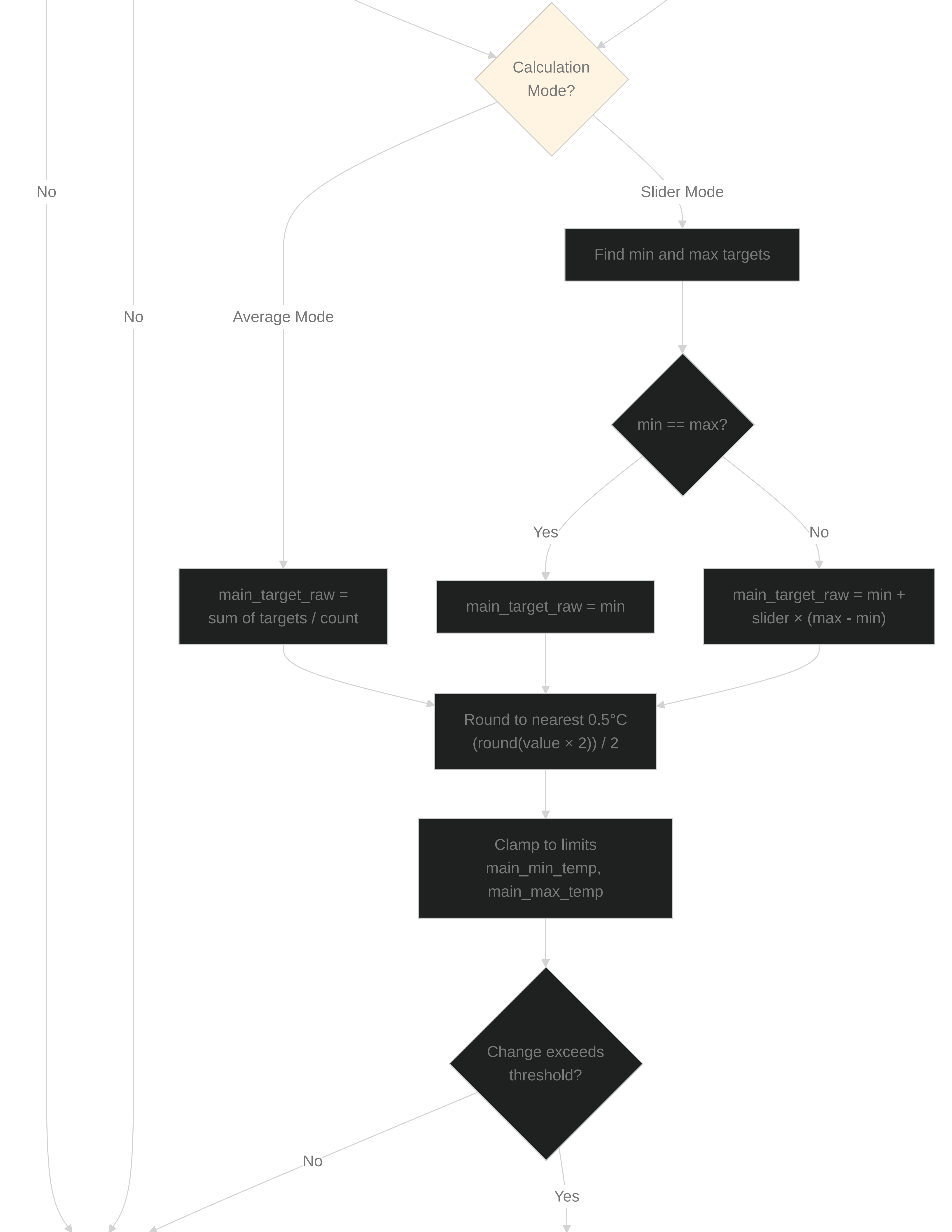
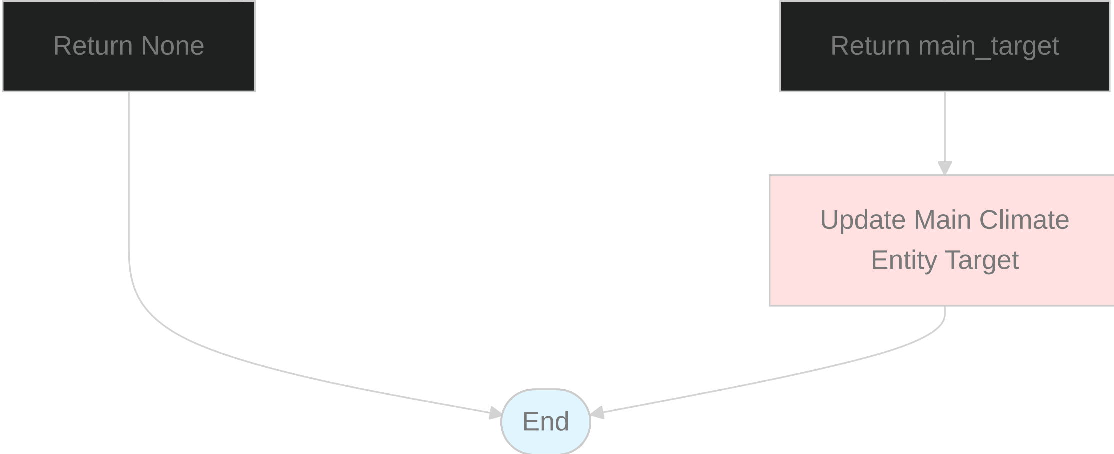


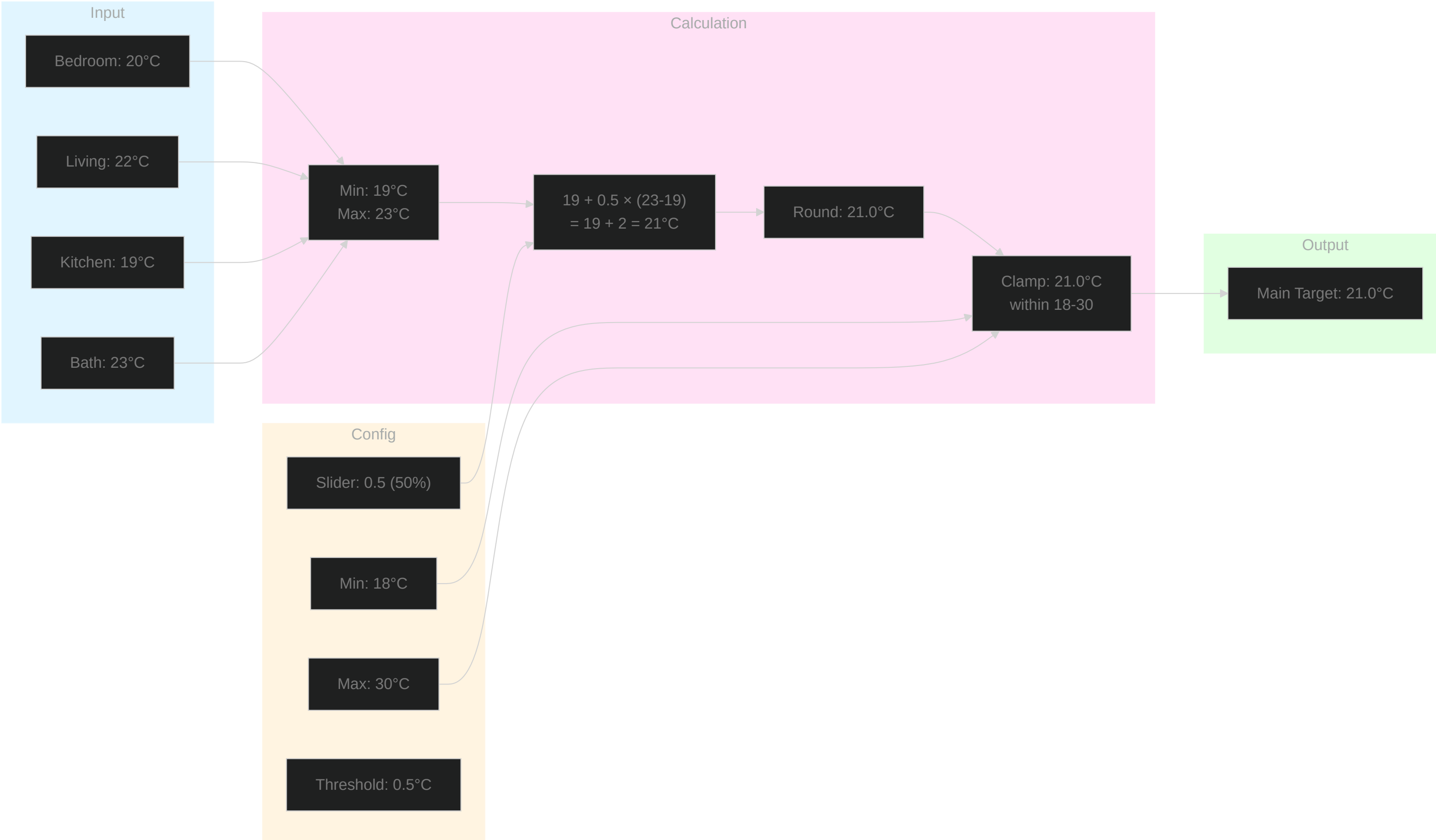
Multizone Integration
Automations
Safety Valve Check Automation
Coordinator Runs every 15s
Execute
Execute
Execute
Trigger
Background Jobs
Calculate Main Target Temperature
Safety Valve Check
Update Valves
Read/Write
Read/Write
Read/Write
Update Target
Open/Close
Core Logic
Core Logic Redis Client
Store/Retrieve
Store/Retrieve
Store/Retrieve
Store/Retrieve
Store/Retrieve
Store/Retrieve
Redis Storage
Global Config
Zone States
Job Queues
Valve Locks
Job Locks
Job Status
HVAC System
HVAC Unit De Dietrich
Heat Water
Heating Pipes
Home Assistant
Main Climate Entity Thermostat Control
Valve Switches Per Room
Control
Control Flow
Heat Rooms
Temperature Sensors Per Room
Dequeue
Dequeue
Read Temps
Climate Zone Subdevices Per Room
Target Change
Update Main Target Temperature Automation
Enqueue Job
Enqueue Job
Queues
Calculate Temp Queue
Update Valves Queue
Main Climate Device Config Entry

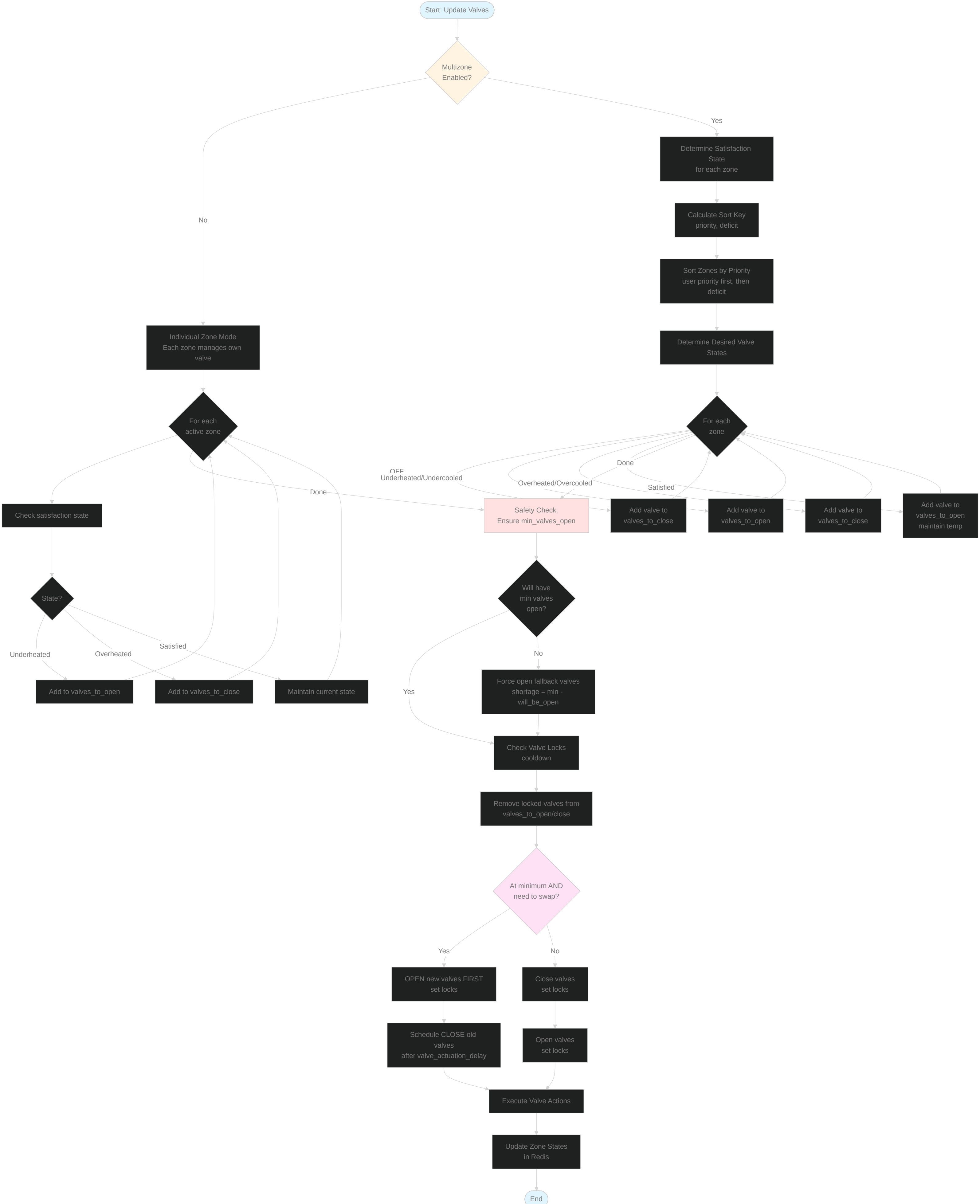


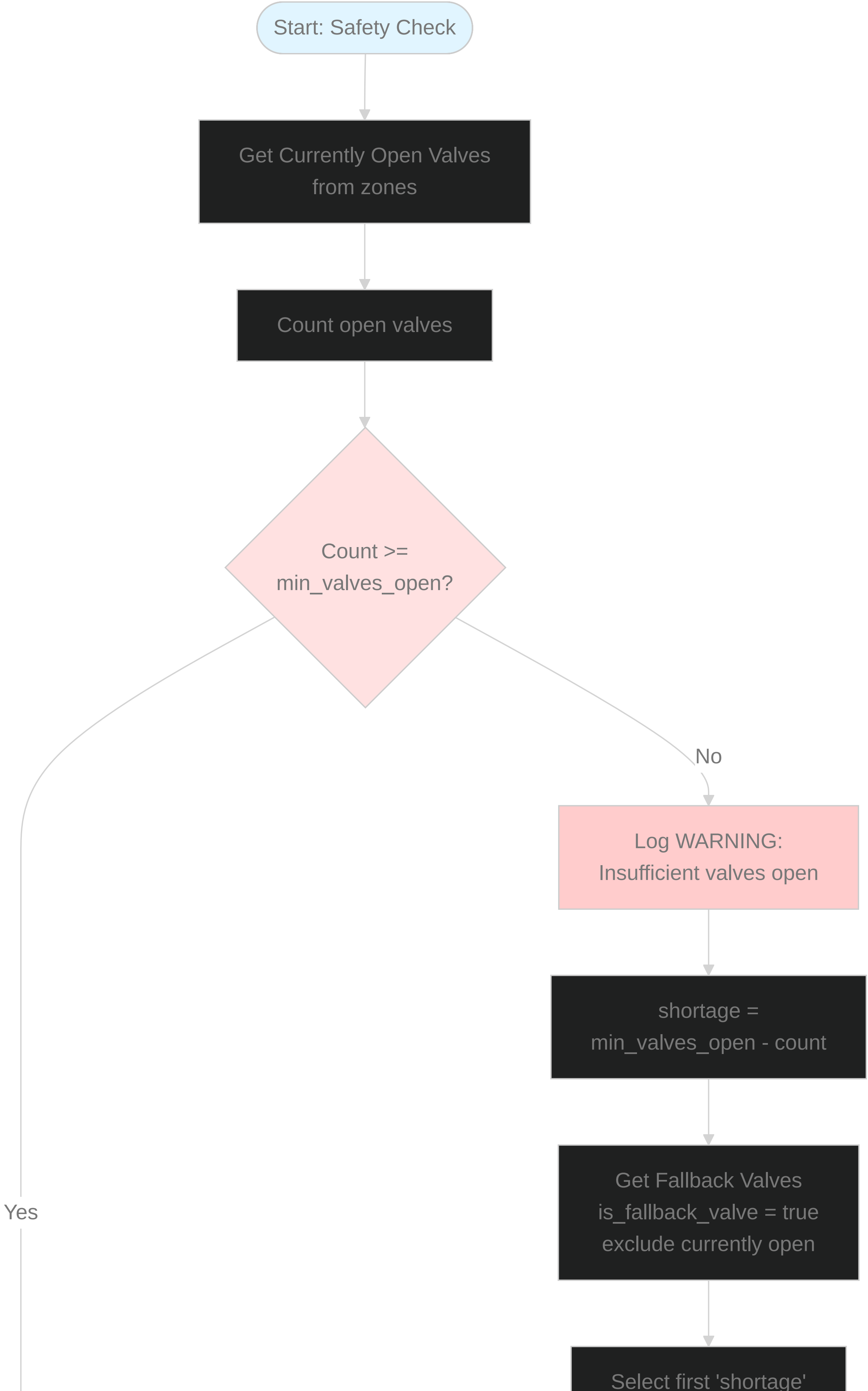
Start: Calculate Main Target
Active Zones Available?
Yes
Filter: Get Active Zones state != OFF
Active Zones Found?
Yes
Filter: Exclude Overheated Zones satisfaction != overheated
Non-Overheated Zones Found?
Yes
No
Use Non-Overheated Zone Targets
Use All Active Zone Targets Fallback


Calculation Mode?
No
No
Average Mode
Slider Mode
Find min and max targets
min == max?
Yes
No
main_target_raw = sum of targets / count
main_target_raw = min
main_target_raw = min + slider × (max - min)
Round to nearest 0.5°C (round(value × 2)) / 2
Clamp to limits main_min_temp, main_max_temp
Change exceeds threshold?
No
Yes

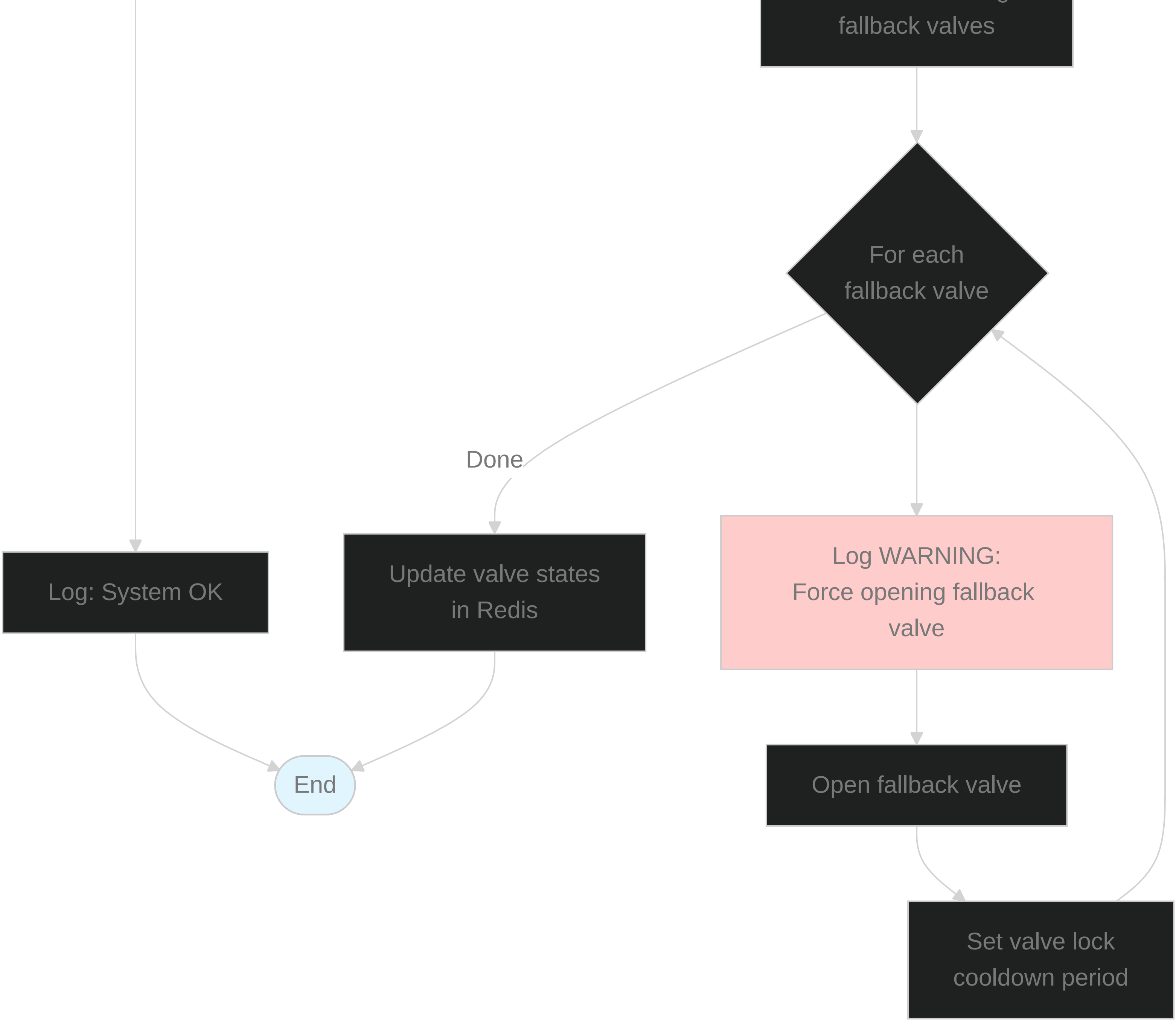

Return None
Return main_target
Update Main Climate Entity Target
End


Input
Bedroom: 20°C
Living: 22°C
Kitchen: 19°C
Bath: 23°C
Calculation
Min: 19°C
Max: 23°C
19 + 0.5 × (23-19)
= 19 + 2 = 21°C
Round: 21.0°C
Clamp: 21.0°C
within 18-30
Output
Main Target: 21.0°C
Config
Slider: 0.5 (50%)
Min: 18°C
Max: 30°C
Threshold: 0.5°C

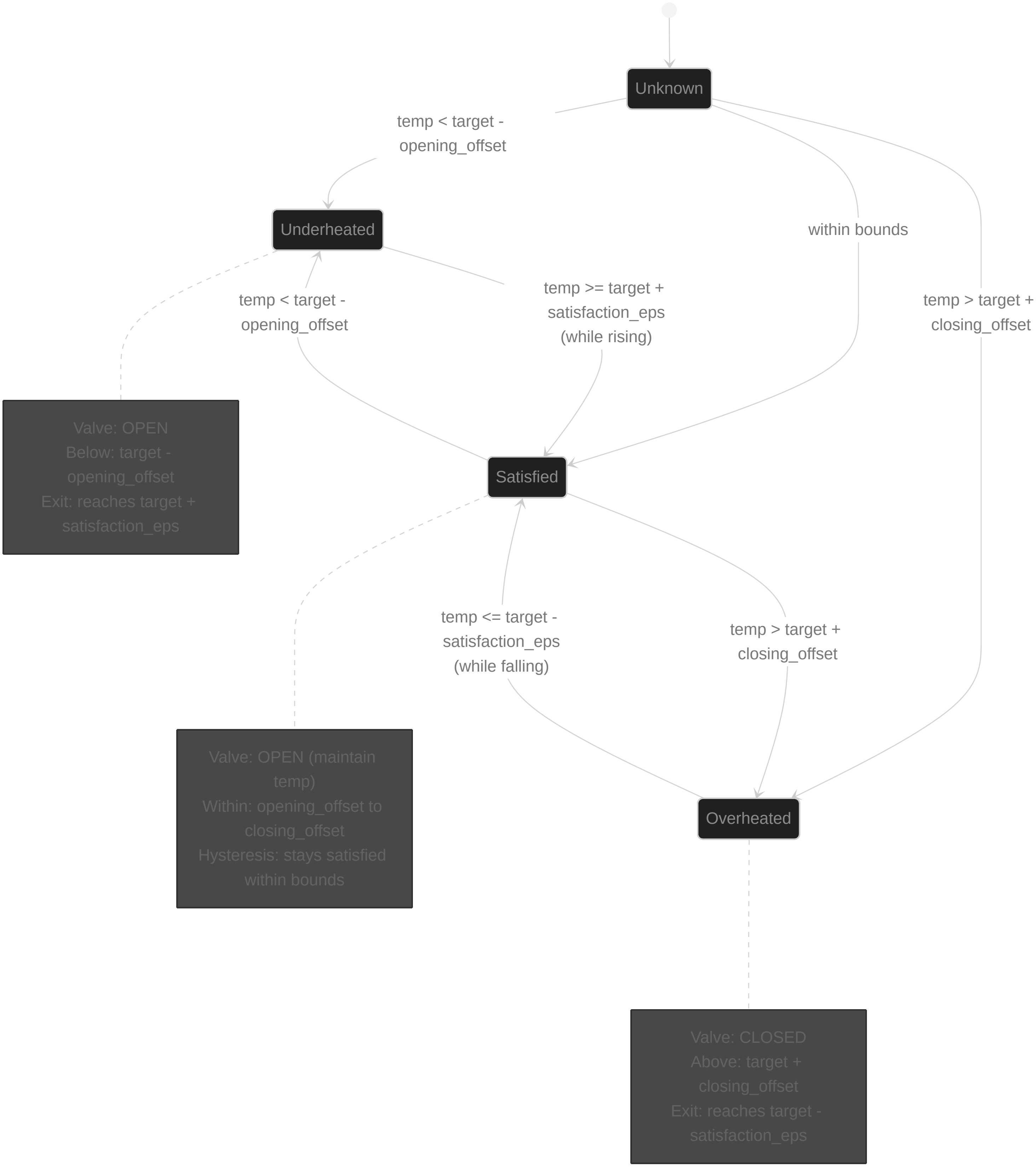

Start: Update Valves
Multizone Enabled?
Yes
No
Determine Satisfaction State for each zone
Calculate Sort Key priority, deficit
Sort Zones by Priority user priority first, then deficit
Determine Desired Valve States
Individual Zone Mode Each zone manages own valve
For each active zone
For each zone
OFF
Underheated/Undercooled
Overheated/Overcooled
Done
Satisfied
Done
Check satisfaction state
Safety Check: Ensure min_valves_open
Add valve to valves_to_close
Add valve to valves_to_open
Add valve to valves_to_close
Add valve to valves_to_open maintain temp
State?
Underheated
Overheated
Satisfied
Add to valves_to_open
Add to valves_to_close
Maintain current state
Will have min valves open?
No
Yes
Force open fallback valves shortage = min - will_be_open
Check Valve Locks cooldown
Remove locked valves from valves_to_open/close
At minimum AND need to swap?
Yes
No
OPEN new valves FIRST set locks
Close valves set locks
Schedule CLOSE old valves after valve_actuation_delay
Open valves set locks
Execute Valve Actions
Update Zone States in Redis
End

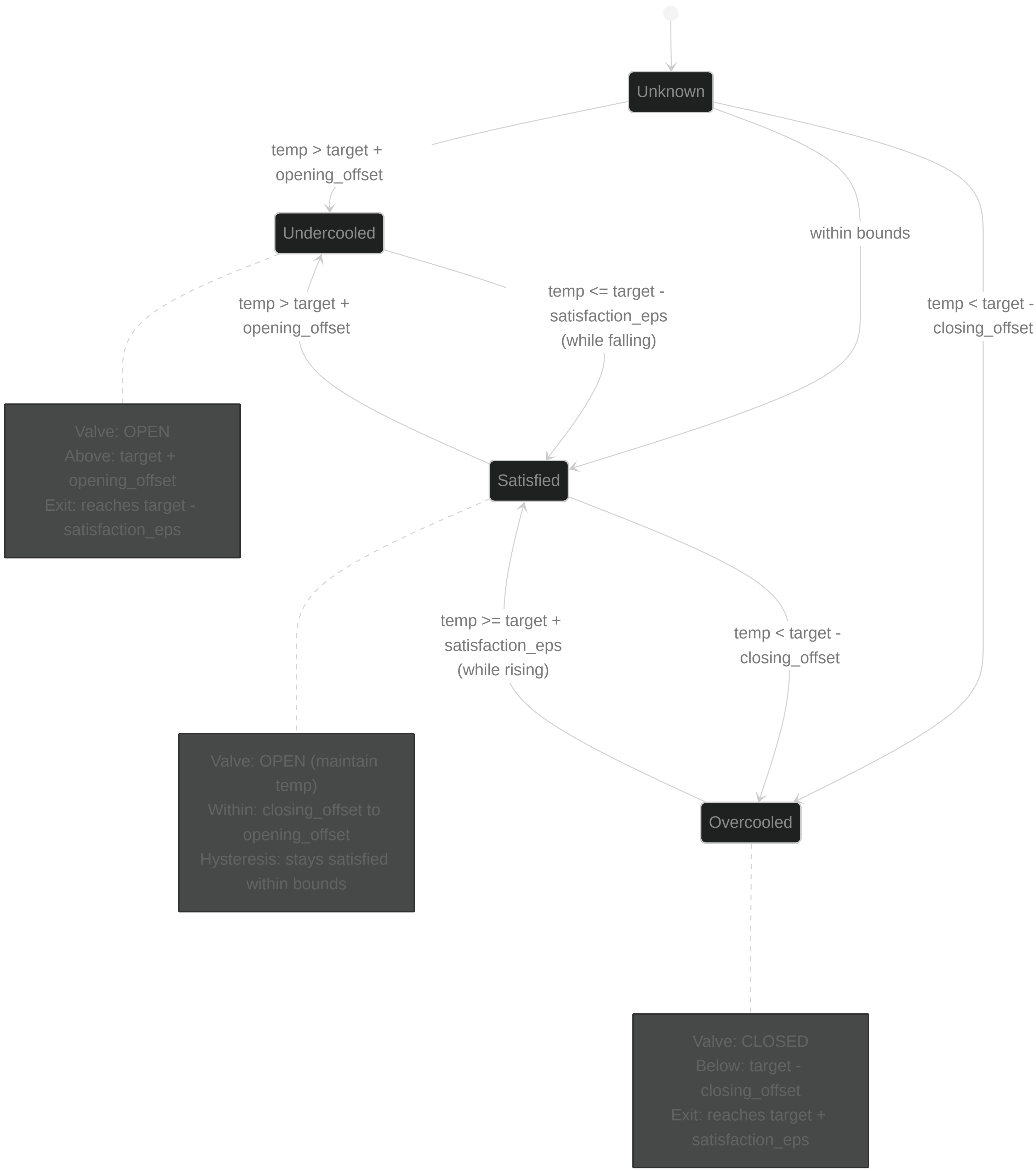


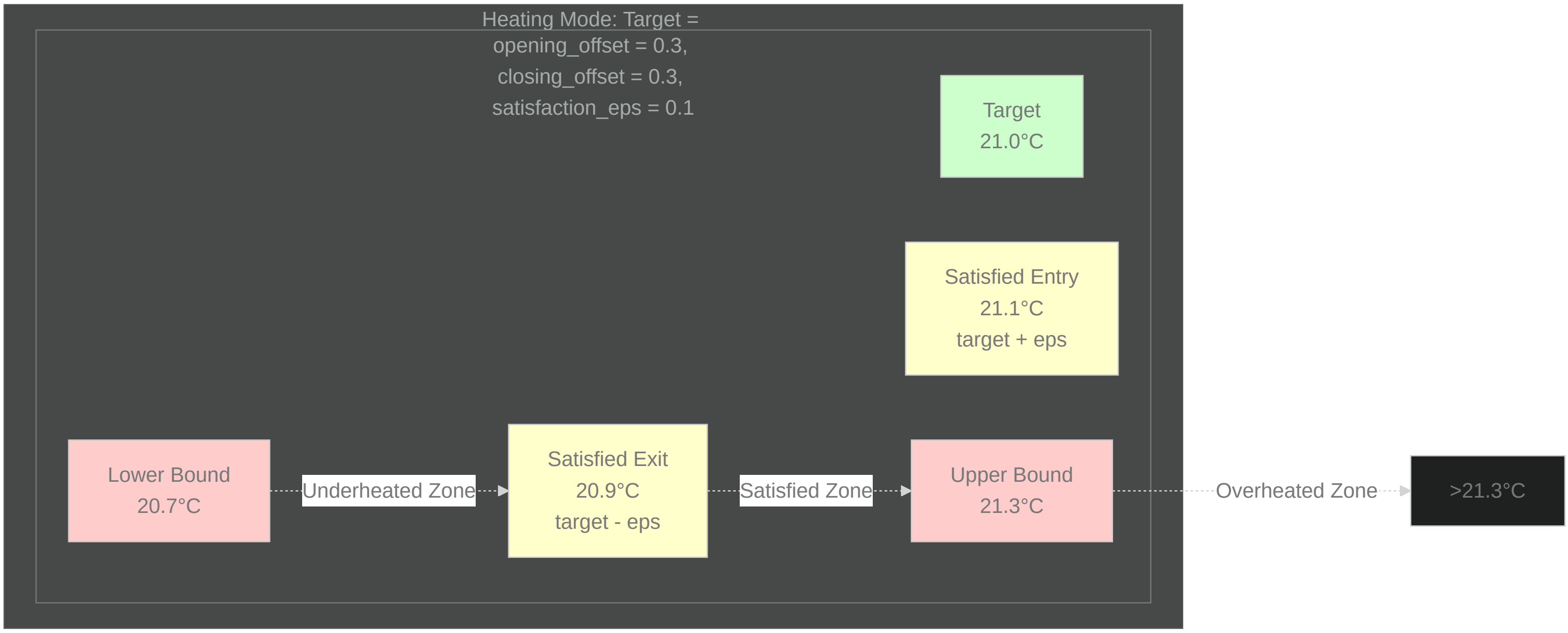
Start: Safety Check
Get Currently Open Valves
from zones
Count open valves
Count >=
min_valves_open?
Yes
No
Log WARNING:
Insufficient valves open
shortage =
min_valves_open - count
Get Fallback Valves
is_fallback_valve = true
exclude currently open
Select first 'shortage'



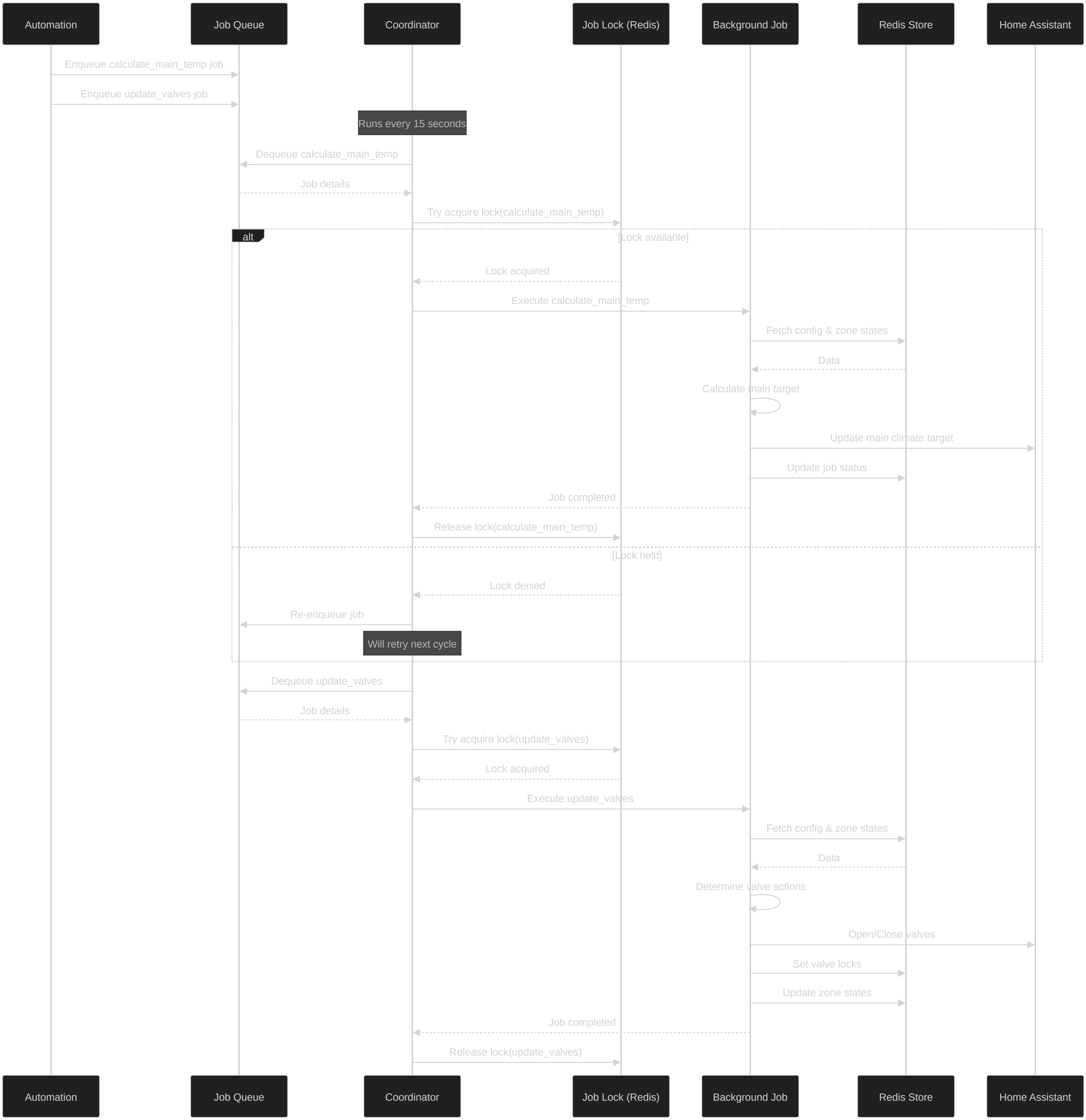
fallback valves
For each fallback valve
Done
Log: System OK
Update valve states in Redis
Log WARNING: Force opening fallback valve
Open fallback valve
End
Set valve lock cooldown period

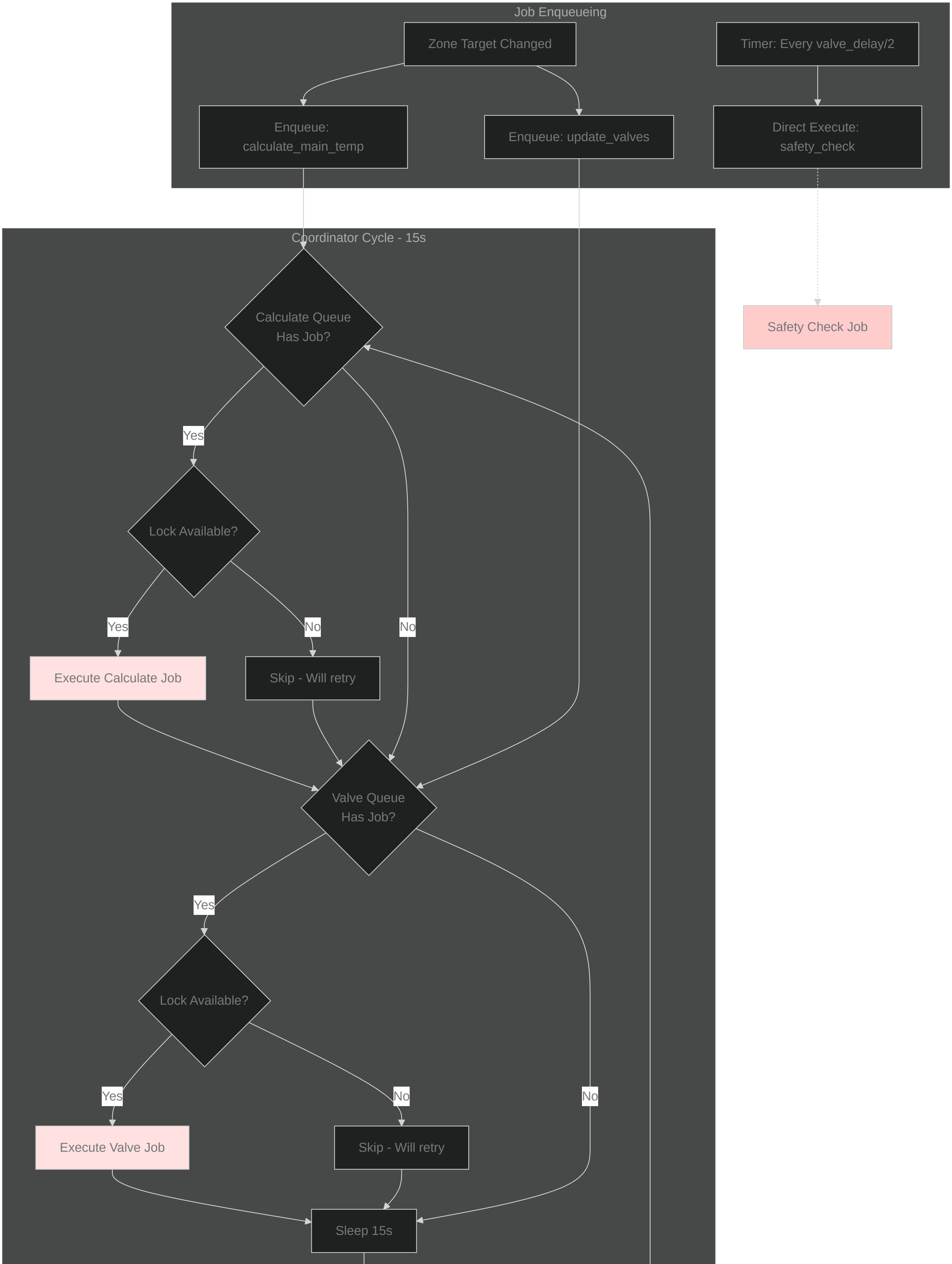

Unknown
temp < target - opening_offset
Underheated
within bounds
temp > target + closing_offset
temp < target - opening_offset
temp >= target + satisfaction_eps (while rising)
Valve: OPEN
Below: target - opening_offset
Exit: reaches target + satisfaction_eps
Satisfied
temp <= target - satisfaction_eps (while falling)
temp > target + closing_offset
Valve: OPEN (maintain temp)
Within: opening_offset to closing_offset
Hysteresis: stays satisfied within bounds
Overheated
Valve: CLOSED
Above: target + closing_offset
Exit: reaches target - satisfaction_eps

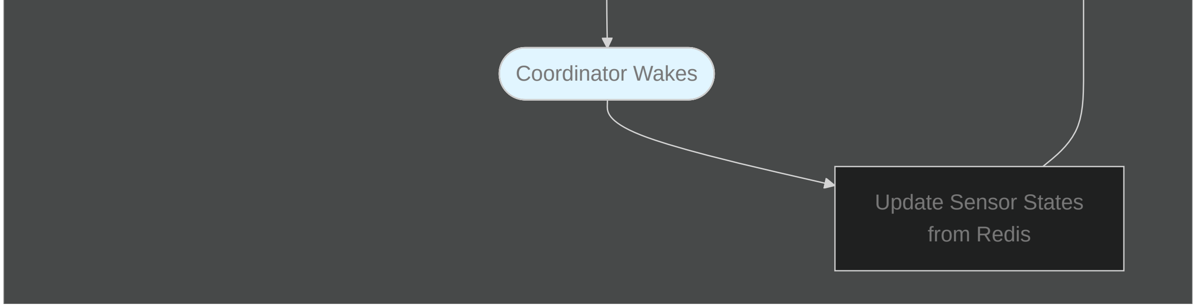

Unknown
temp > target + opening_offset
Undercooled
within bounds
temp <= target - satisfaction_eps (while falling)
temp > target + opening_offset
temp < target - closing_offset
Valve: OPEN
Above: target + opening_offset
Exit: reaches target - satisfaction_eps
Satisfied
temp >= target + satisfaction_eps (while rising)
temp < target - closing_offset
Valve: OPEN (maintain temp)
Within: closing_offset to opening_offset
Hysteresis: stays satisfied within bounds
Overcooled
Valve: CLOSED
Below: target - closing_offset
Exit: reaches target + satisfaction_eps



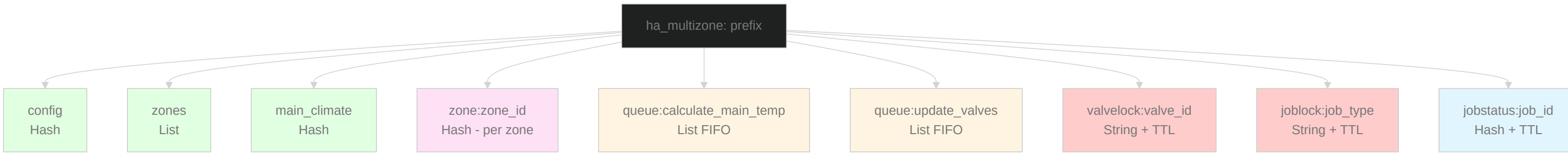
Heating Mode: Target =
opening_offset = 0.3,
closing_offset = 0.3,
satisfaction_eps = 0.1
Target
21.0°C
Satisfied Entry
21.1°C
target + eps
Lower Bound
20.7°C
Underheated Zone
Satisfied Exit
20.9°C
target - eps
Satisfied Zone
Upper Bound
21.3°C
Overheated Zone
>21.3°C

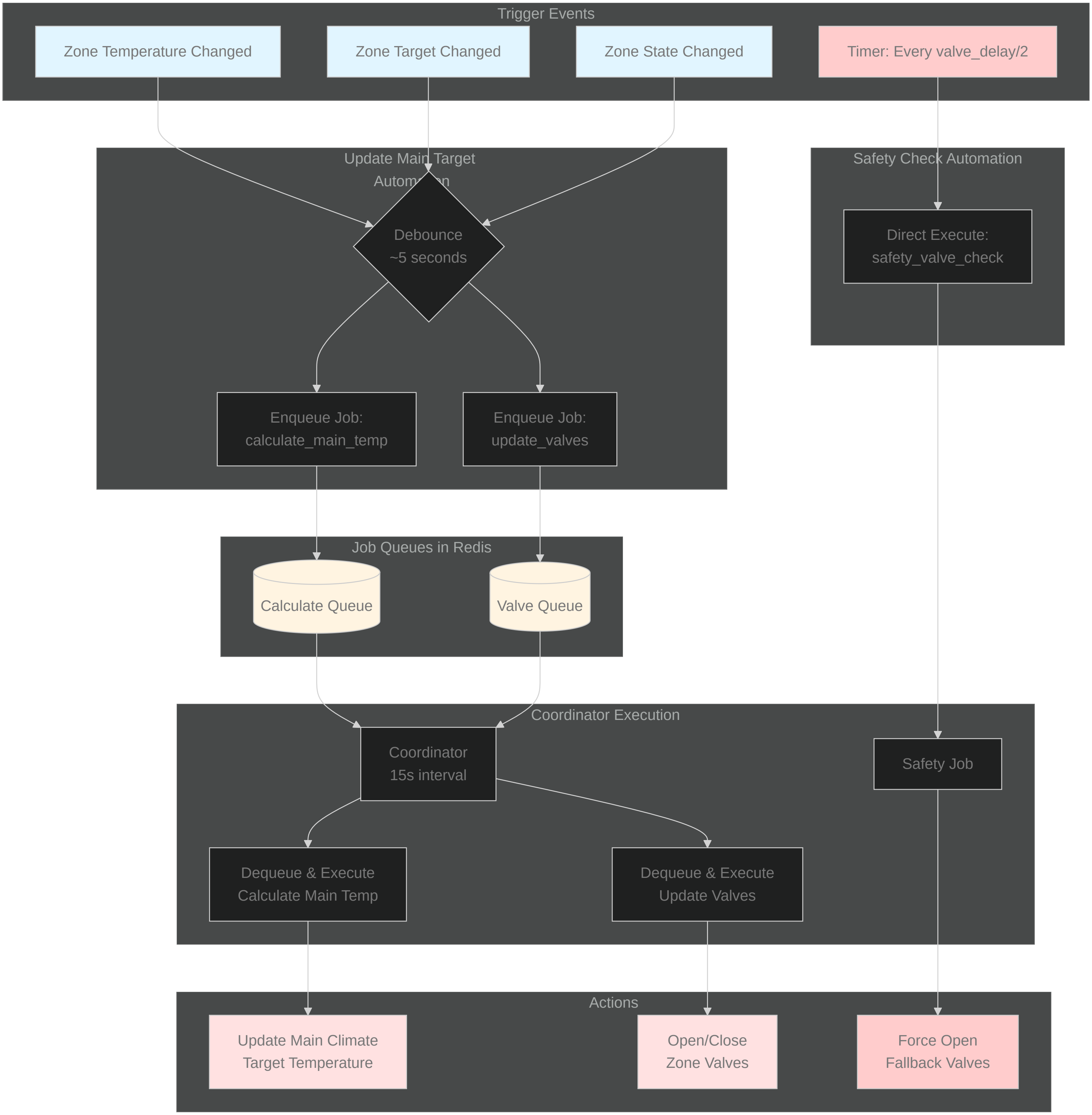

Automation
Job Queue
Coordinator
Job Lock (Redis)
Background Job
Redis Store
Home Assistant
Enqueue calculate_main_temp job
Enqueue update_valves job
Runs every 15 seconds
Dequeue calculate_main_temp
Job details
Try acquire lock(calculate_main_temp)
alt
[Lock available]
Lock acquired
Execute calculate_main_temp
Fetch config & zone states
Data
Calculate main target
Update main climate target
Update job status
Job completed
Release lock(calculate_main_temp)
[Lock held]
Lock denied
Re-enqueue job
Will retry next cycle
Dequeue update_valves
Job details
Try acquire lock(update_valves)
Lock acquired
Execute update_valves
Fetch config & zone states
Data
Determine valve actions
Open/Close valves
Set valve locks
Update zone states
Job completed
Release lock(update_valves)
Automation
Job Queue
Coordinator
Job Lock (Redis)
Background Job
Redis Store
Home Assistant

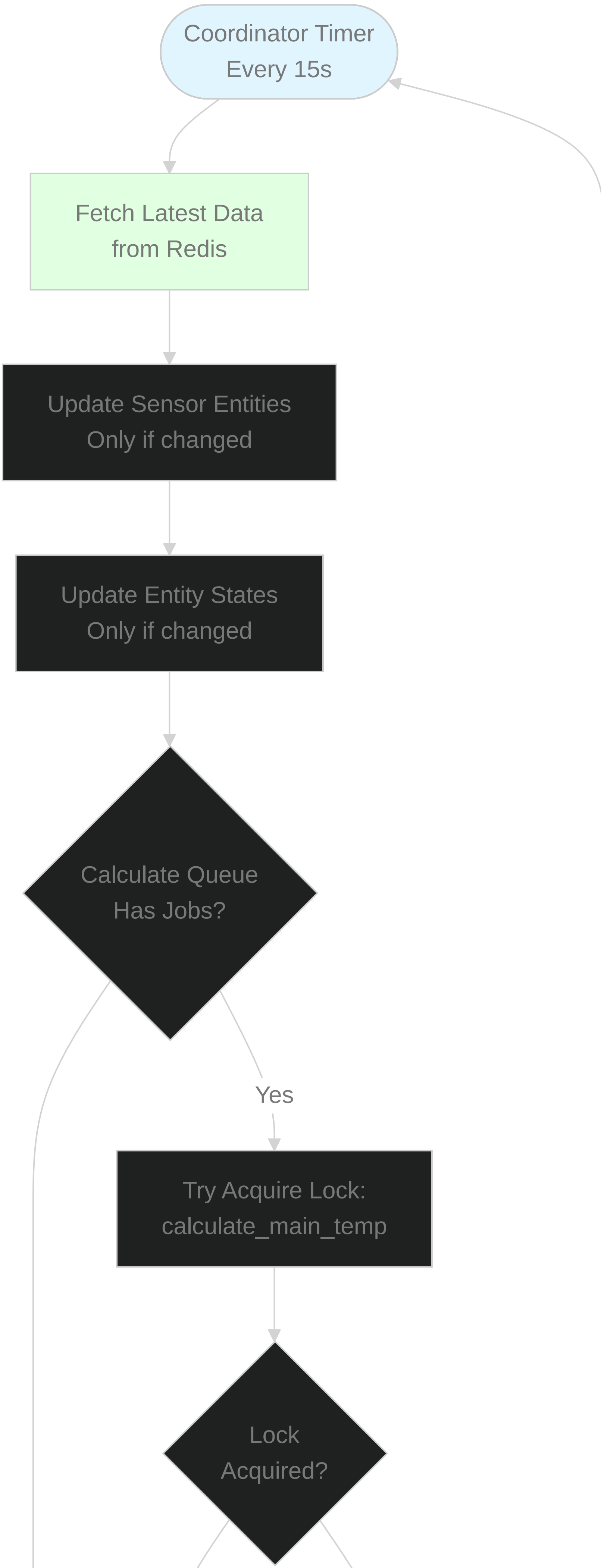

Job Enqueueing
Zone Target Changed
Timer: Every valve_delay/2
Enqueue: calculate_main_temp
Enqueue: update_valves
Direct Execute: safety_check
Safety Check Job
Coordinator Cycle - 15s
Calculate Queue Has Job?
Yes
Lock Available?
Yes
No
No
Execute Calculate Job
Skip - Will retry
Valve Queue Has Job?
Yes
Lock Available?
Yes
No
No
Execute Valve Job
Skip - Will retry
Sleep 15s


Coordinator Wakes
Update Sensor States
from Redis

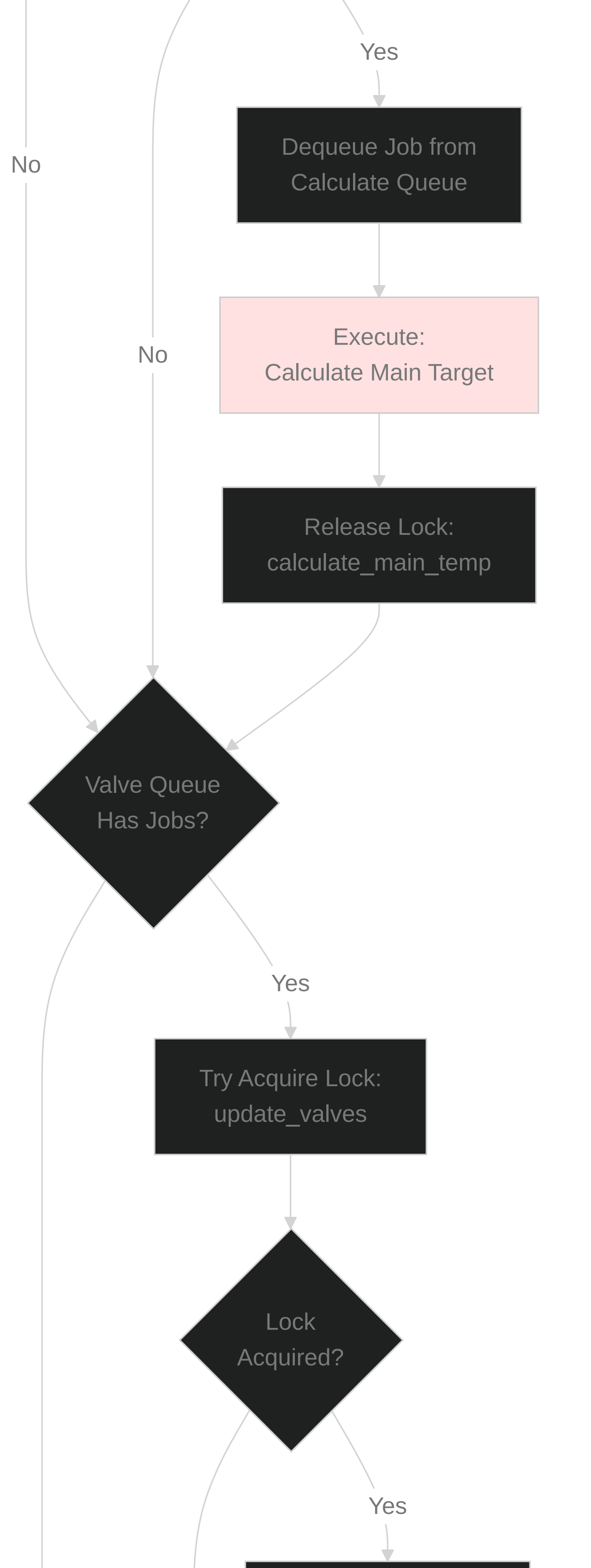

CONFIG
string main_climate_entity_id
float main_target_all_zones_satisfied
boolean use_average_mode
int min_valves_open
float main_min_temp
float main_max_temp
float main_change_threshold
int valve_actuation_delay
int coordinator_interval
float satisfaction_eps
manages
ZONES
array zone_ids
contains
ZONE_STATE
string id
string name
string temperature_sensor_entity_id
string valve_switch_entity_id
float current_temperature
float target_temperature
string state
string satisfaction
string valve_state
float opening_offset
float closing_offset
boolean is_fallback_valve
int priority
timestamp last_updated
MAIN_CLIMATE
string entity_id
float current_temperature
float target_temperature
float outdoor_temperature
string hvac_mode
string hvac_action
boolean multizone_enabled
timestamp last_updated
JOB_QUEUES
string queue_calculate_main_temp
string queue_update_valves
contains
QUEUE_ENTRY
string job_id
string job_type
timestamp enqueued_at
json parameters
VALVE_LOCKS
string valve_id
timestamp locked_until
string reason
JOB_LOCKS
string job_type
timestamp acquired_at
string acquired_by
JOB_STATUS
string job_id
string job_type
string status
timestamp started_at
timestamp completed_at
int duration_ms
int actions_taken
json result


ha_multizone: prefix
config
Hash
zones
List
main_climate
Hash
zone:zone_id
Hash - per zone
queue:calculate_main_temp
List FIFO
queue:update_valves
List FIFO
valvelock:valve_id
String + TTL
joblock:job_type
String + TTL
jobstatus:job_id
Hash + TTL

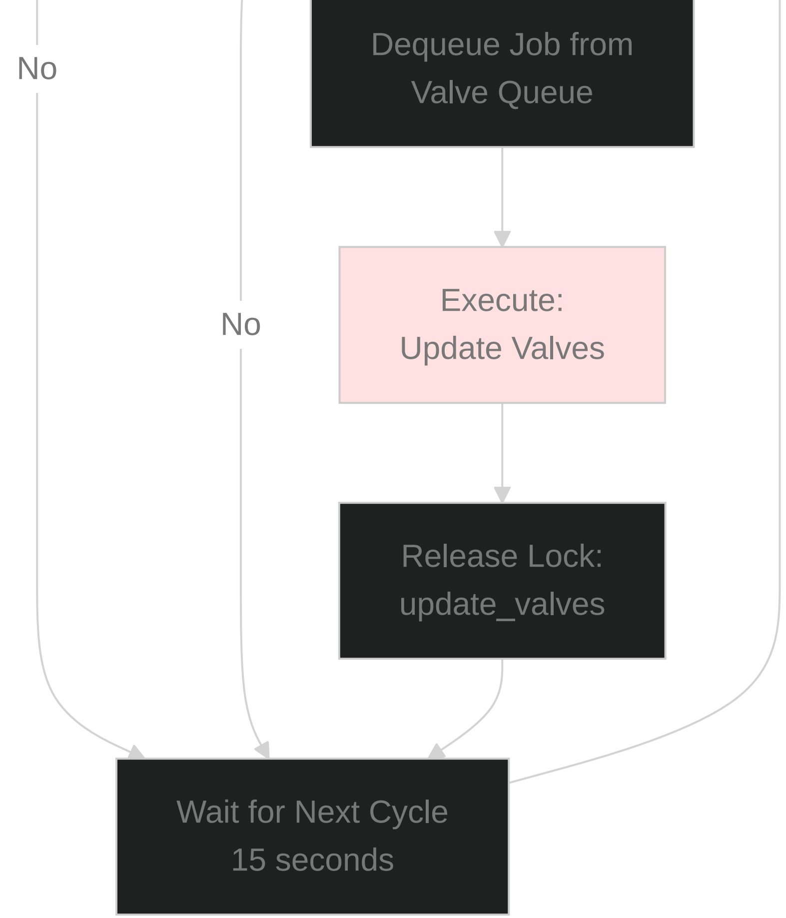

Trigger Events
Zone Temperature Changed
Zone Target Changed
Zone State Changed
Timer: Every valve_delay/2
Update Main Target Automation
Debounce ~5 seconds
Enqueue Job: calculate_main_temp
Enqueue Job: update_valves
Safety Check Automation
Direct Execute: safety_valve_check
Job Queues in Redis
Calculate Queue
Valve Queue
Coordinator Execution
Coordinator 15s interval
Safety Job
Dequeue & Execute Calculate Main Temp
Dequeue & Execute Update Valves
Actions
Update Main Climate Target Temperature
Open/Close Zone Valves
Force Open Fallback Valves

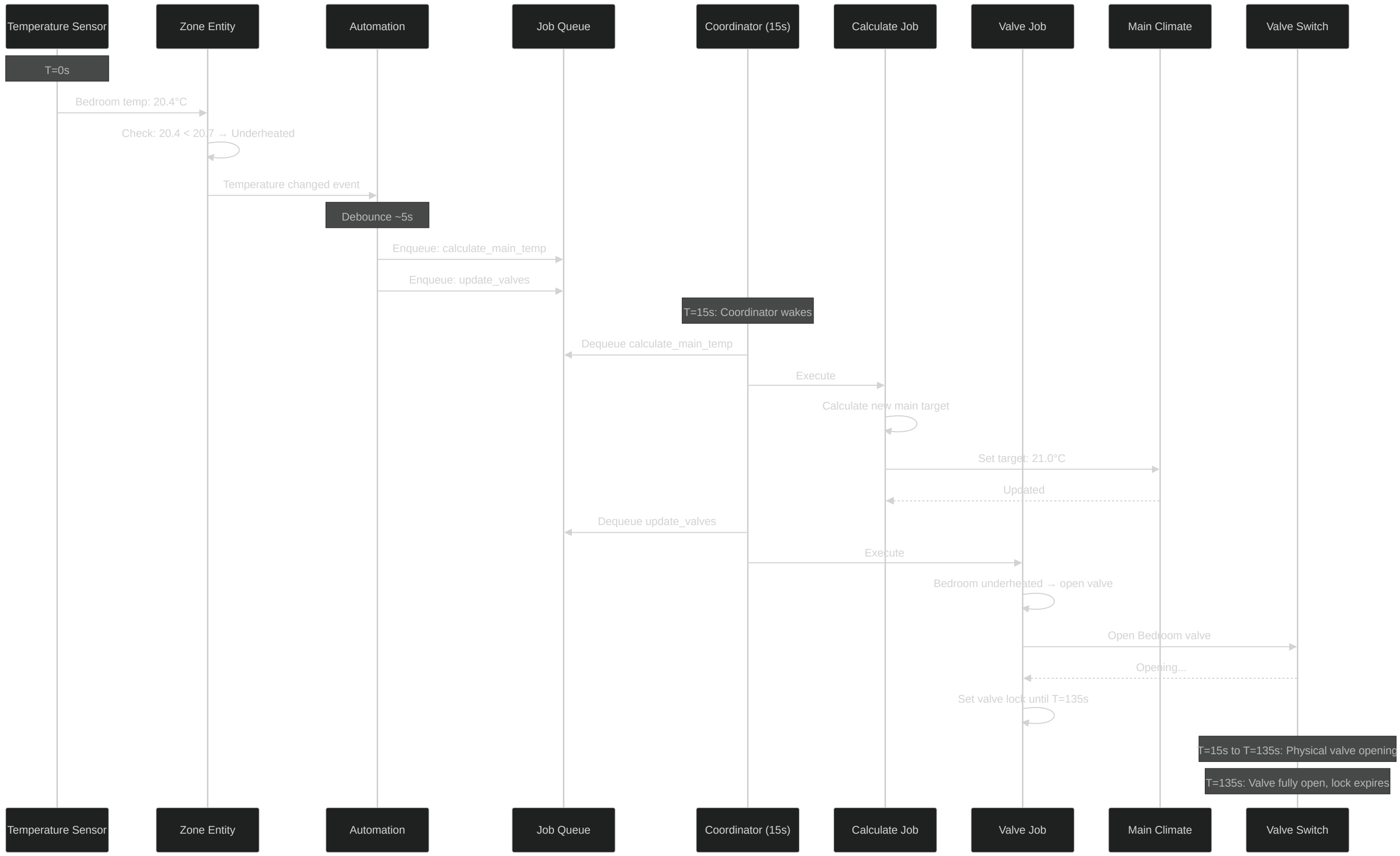


Coordinator Timer
Every 15s
Fetch Latest Data
from Redis
Update Sensor Entities
Only if changed
Update Entity States
Only if changed
Calculate Queue
Has Jobs?
Yes
Try Acquire Lock:
calculate_main_temp
Lock
Acquired?

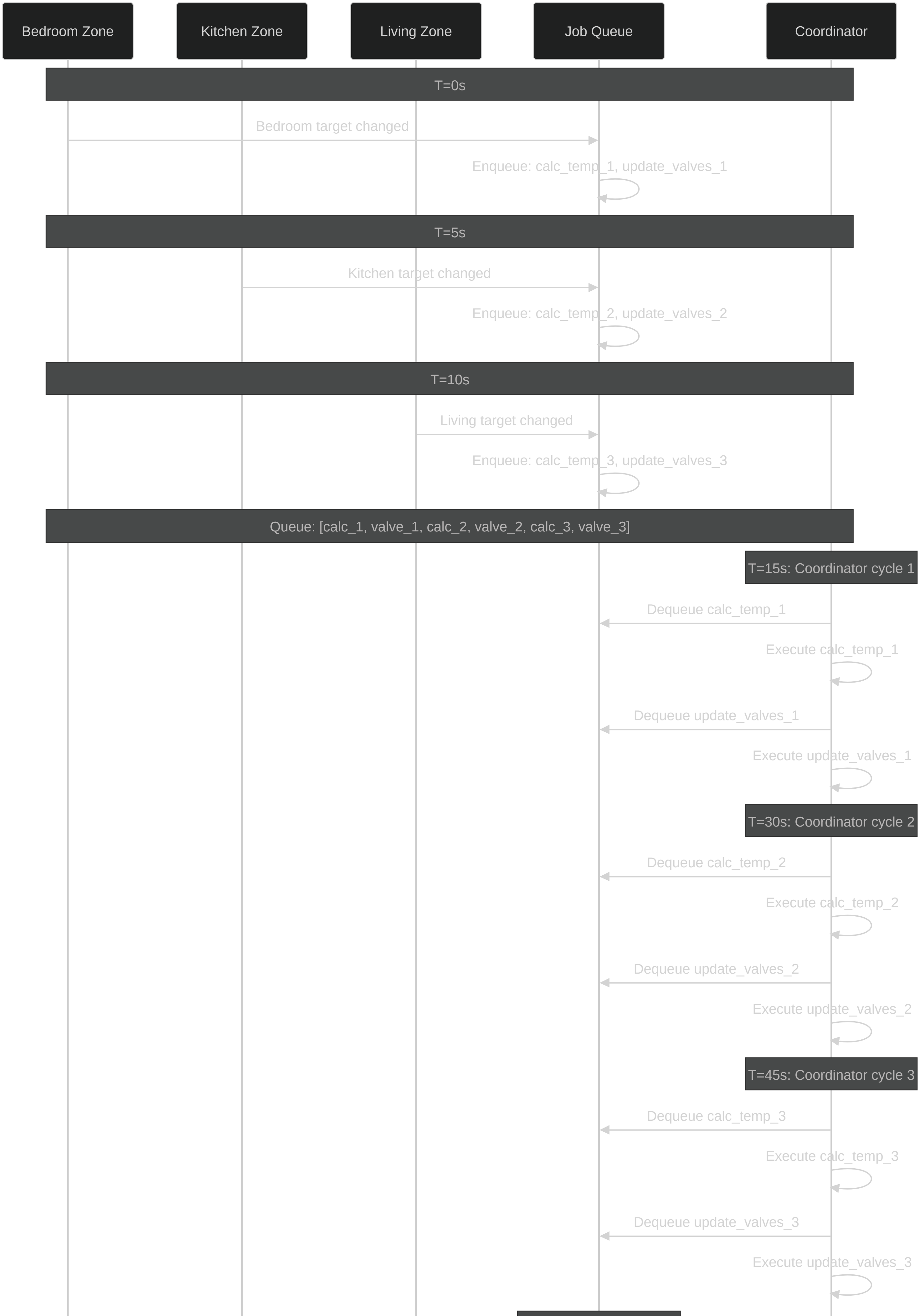


Yes
No
Dequeue Job from
Calculate Queue
No
Execute:
Calculate Main Target
Release Lock:
calculate_main_temp
Valve Queue
Has Jobs?
Yes
Try Acquire Lock:
update_valves
Lock
Acquired?
Yes

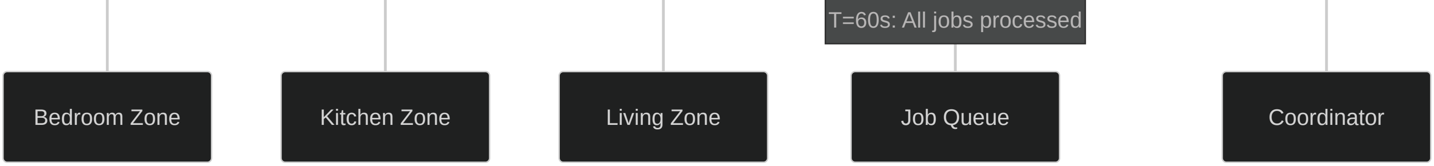

No
Dequeue Job from
Valve Queue
No
Execute:
Update Valves
Release Lock:
update_valves
Wait for Next Cycle
15 seconds

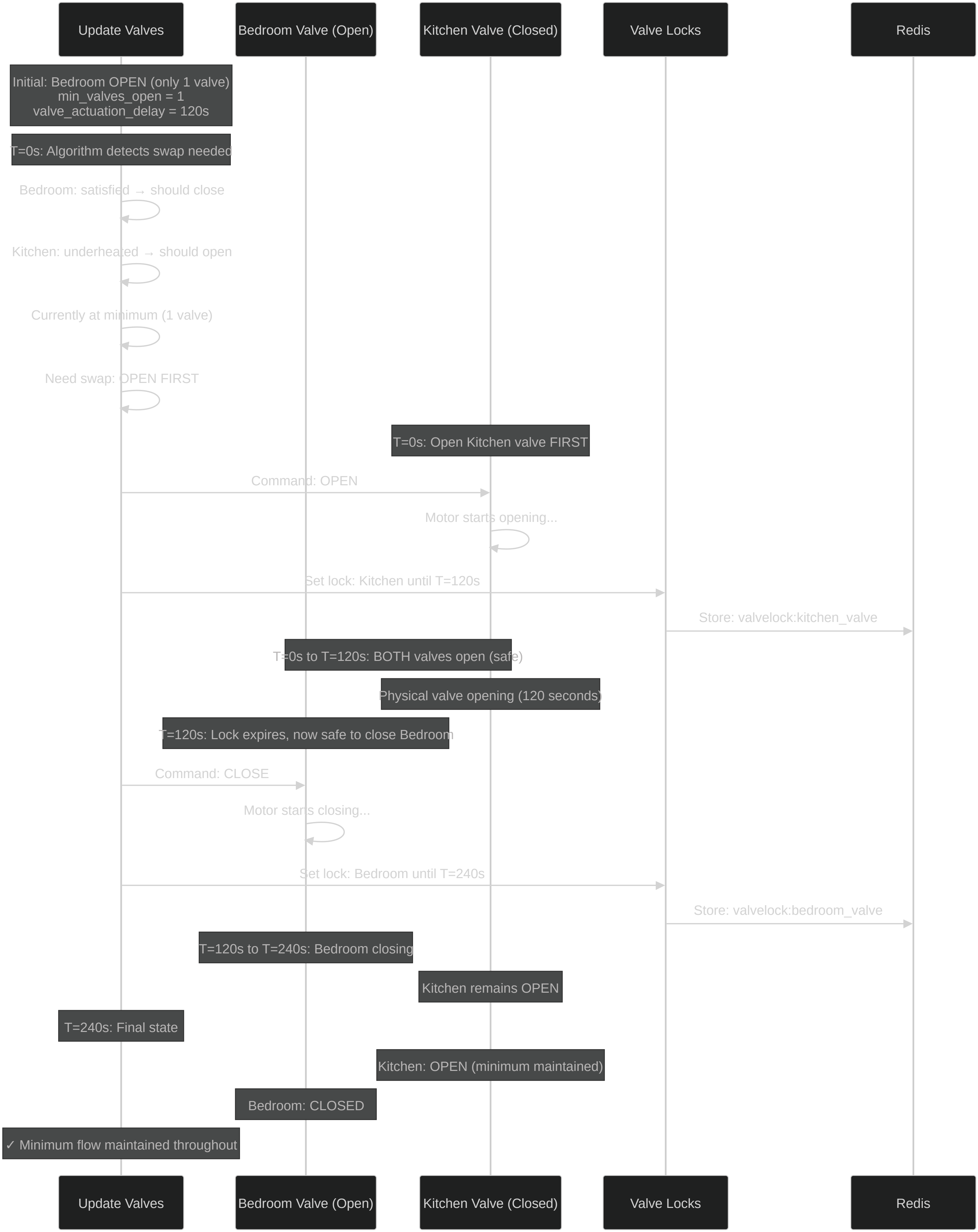


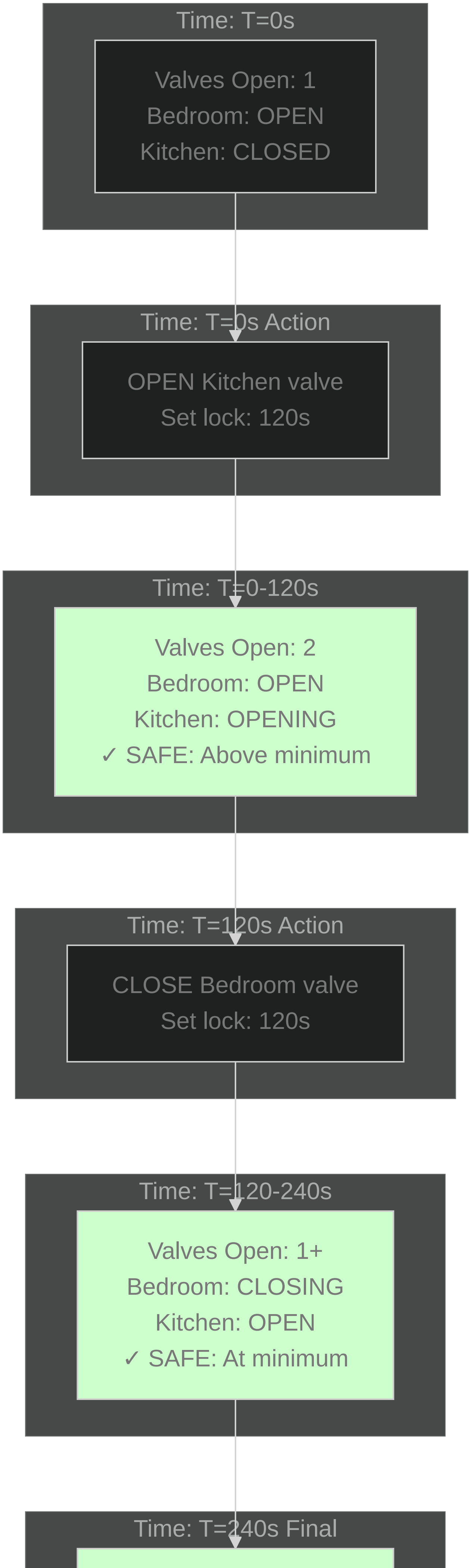
Temperature Sensor
Zone Entity
Automation
Job Queue
Coordinator (15s)
Calculate Job
Valve Job
Main Climate
Valve Switch
T=0s
Bedroom temp: 20.4°C
Check: 20.4 < 20.7 → Underheated
Temperature changed event
Debounce ~5s
Enqueue: calculate_main_temp
Enqueue: update_valves
T=15s: Coordinator wakes
Dequeue calculate_main_temp
Execute
Calculate new main target
Set target: 21.0°C
Updated
Dequeue update_valves
Execute
Bedroom underheated → open valve
Open Bedroom valve
Opening...
Set valve lock until T=135s
T=15s to T=135s: Physical valve opening
T=135s: Valve fully open, lock expires



Bedroom Zone
Kitchen Zone
Living Zone
Job Queue
Coordinator
T=0s
Bedroom target changed
Enqueue: calc_temp_1, update_valves_1
T=5s
Kitchen target changed
Enqueue: calc_temp_2, update_valves_2
T=10s
Living target changed
Enqueue: calc_temp_3, update_valves_3
Queue: [calc_1, valve_1, calc_2, valve_2, calc_3, valve_3]
T=15s: Coordinator cycle 1
Dequeue calc_temp_1
Execute calc_temp_1
Dequeue update_valves_1
Execute update_valves_1
T=30s: Coordinator cycle 2
Dequeue calc_temp_2
Execute calc_temp_2
Dequeue update_valves_2
Execute update_valves_2
T=45s: Coordinator cycle 3
Dequeue calc_temp_3
Execute calc_temp_3
Dequeue update_valves_3
Execute update_valves_3

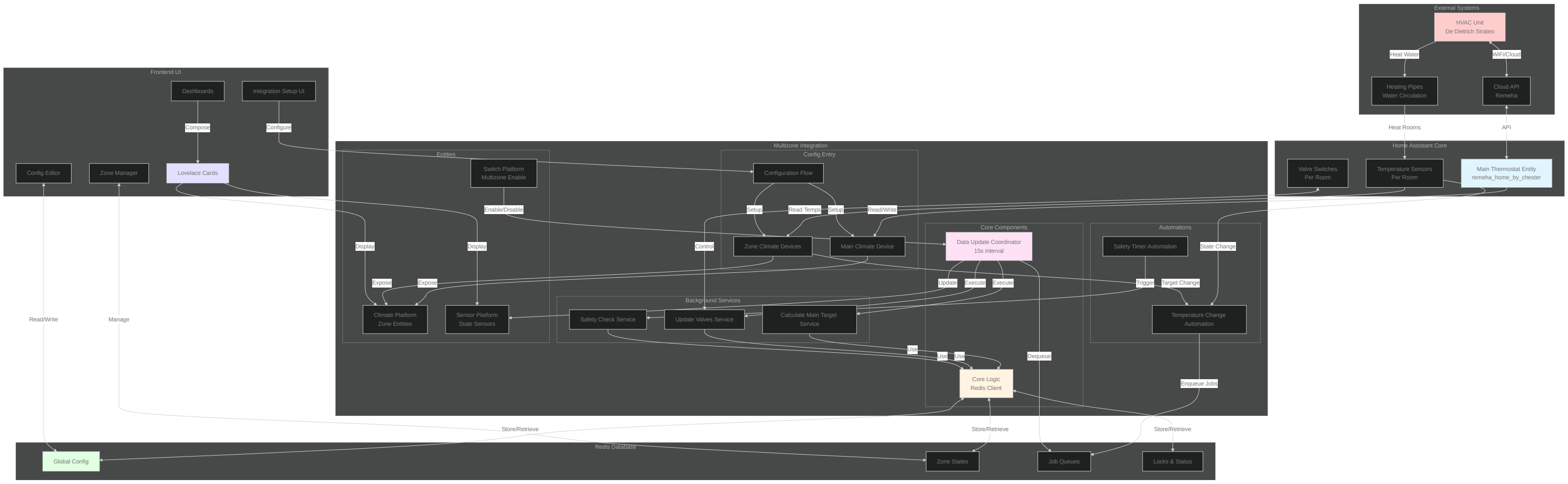

T=60s: All jobs processed
Bedroom Zone
Kitchen Zone
Living Zone
Job Queue
Coordinator


Update Valves
Bedroom Valve (Open)
Kitchen Valve (Closed)
Valve Locks
Redis
Initial: Bedroom OPEN (only 1 valve)
min_valves_open = 1
valve_actuation_delay = 120s
T=0s: Algorithm detects swap needed
Bedroom: satisfied → should close
Kitchen: underheated → should open
Currently at minimum (1 valve)
Need swap: OPEN FIRST
T=0s: Open Kitchen valve FIRST
Command: OPEN
Motor starts opening...
Set lock: Kitchen until T=120s
Store: valvelock:kitchen_valve
T=0s to T=120s: BOTH valves open (safe)
Physical valve opening (120 seconds)
T=120s: Lock expires, now safe to close Bedroom
Command: CLOSE
Motor starts closing...
Set lock: Bedroom until T=240s
Store: valvelock:bedroom_valve
T=120s to T=240s: Bedroom closing
Kitchen remains OPEN
T=240s: Final state
Kitchen: OPEN (minimum maintained)
Bedroom: CLOSED
✓ Minimum flow maintained throughout
Update Valves
Bedroom Valve (Open)
Kitchen Valve (Closed)
Valve Locks
Redis

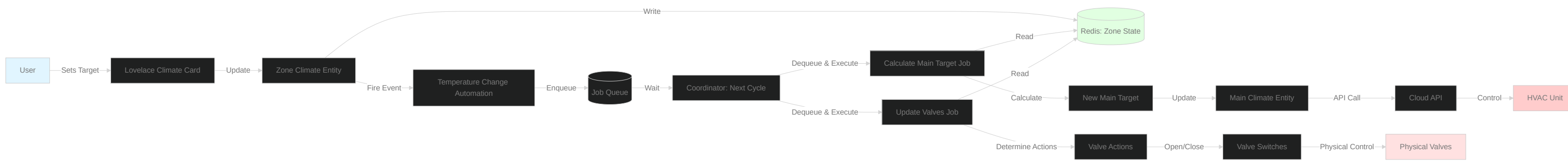


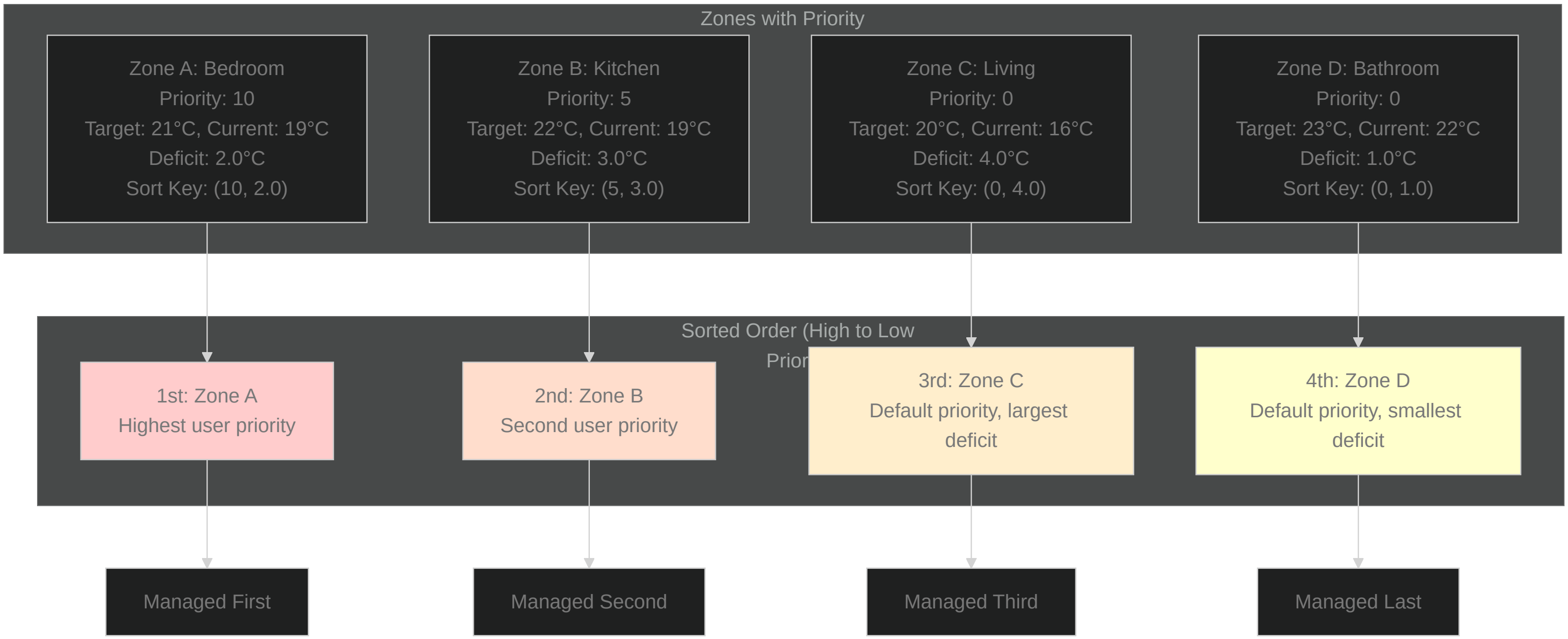
Time: T=0s
Valves Open: 1
Bedroom: OPEN
Kitchen: CLOSED
Time: T=0s Action
OPEN Kitchen valve
Set lock: 120s
Time: T=0-120s
Valves Open: 2
Bedroom: OPEN
Kitchen: OPENING
✓ SAFE: Above minimum
Time: T=120s Action
CLOSE Bedroom valve
Set lock: 120s
Time: T=120-240s
Valves Open: 1+
Bedroom: CLOSING
Kitchen: OPEN
✓ SAFE: At minimum
Time: T=240s Final

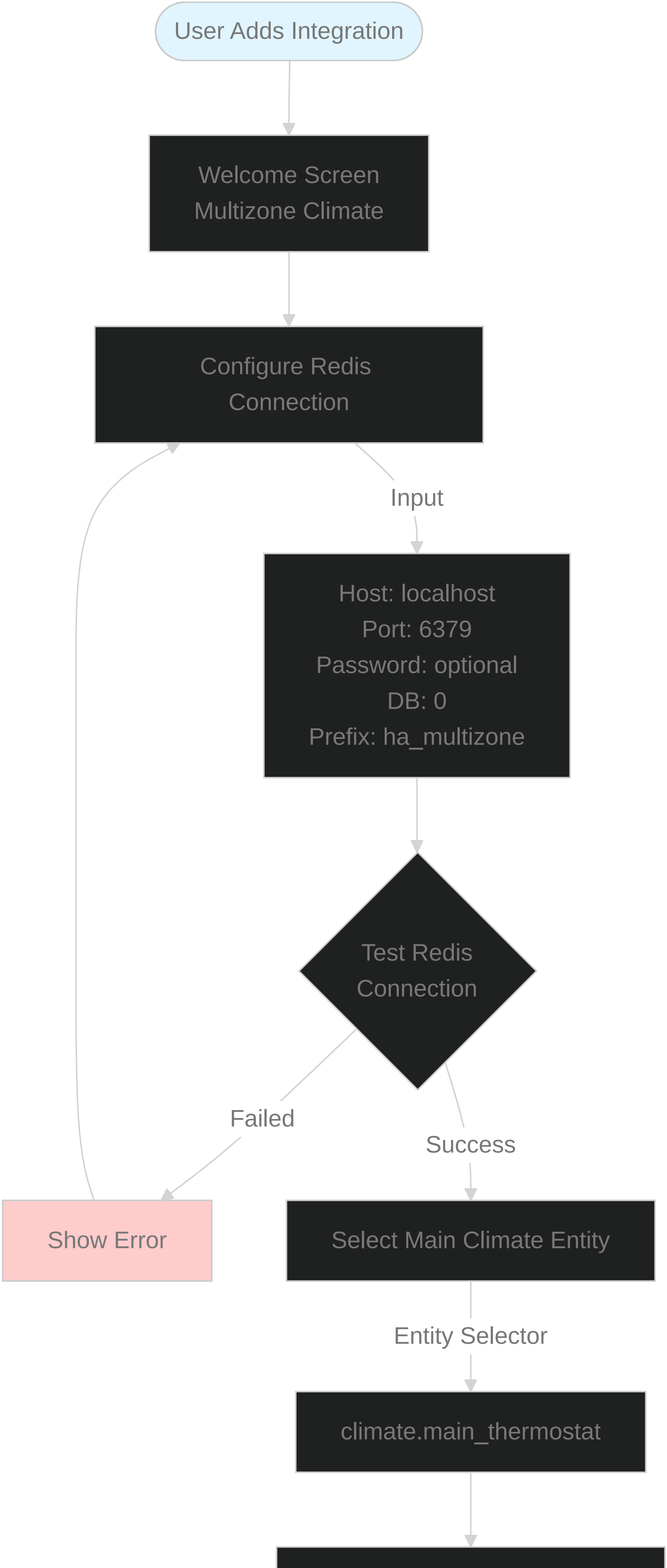
Valves Open: 1
Bedroom: CLOSED
Kitchen: OPEN
✓ SAFE: At minimum

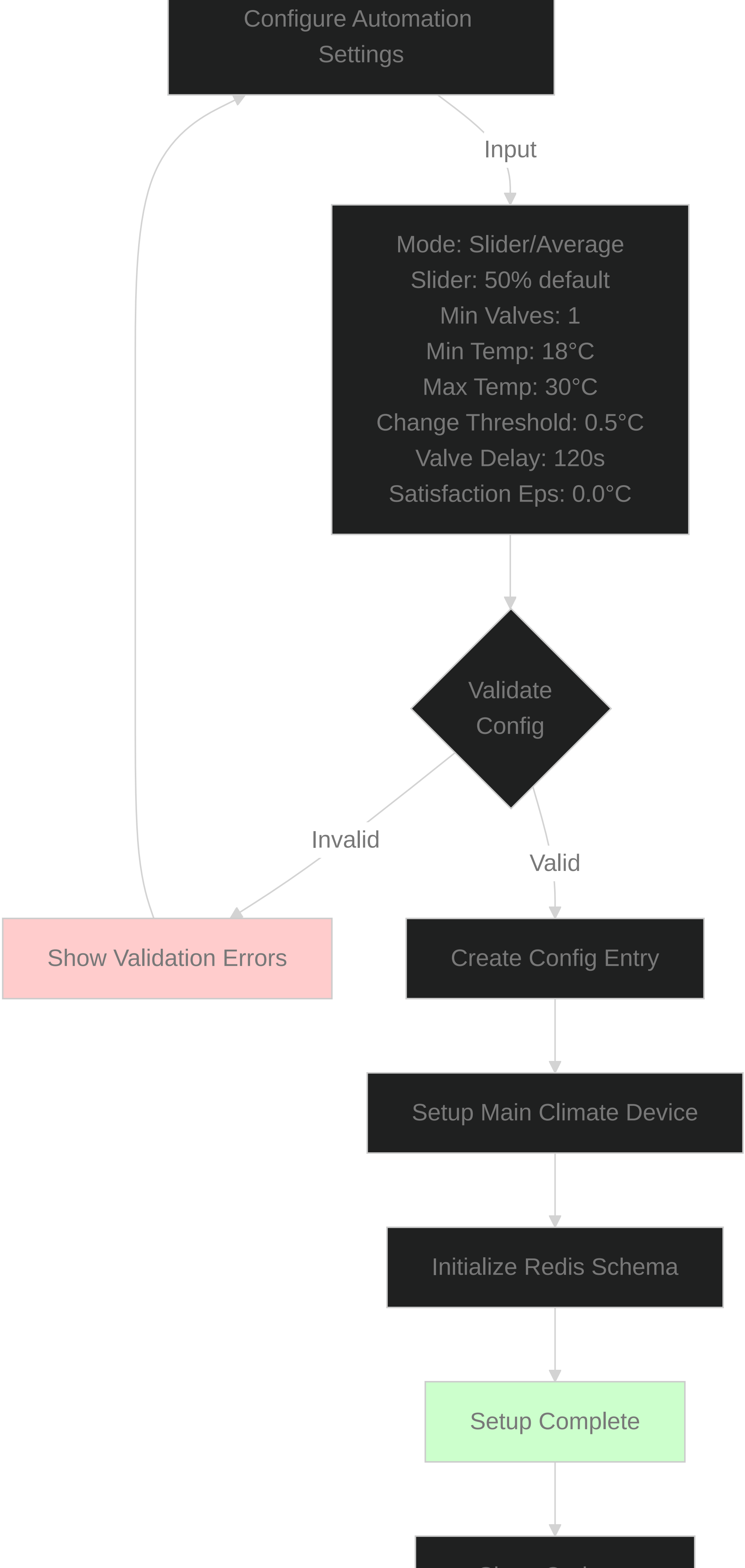


External Systems
HVAC Unit
De Dietrich Strateo
Heat Water
WiFi/Cloud
Heating Pipes
Water Circulation
Cloud API
Remeha
Heat Rooms
API
Home Assistant Core
Valve Switches
Per Room
Temperature Sensors
Per Room
Main Thermostat Entity
remeha_home_by_chester
Frontend UI
Dashboards
Integration Setup UI
Compose
Configure
Config Editor
Zone Manager
Lovelace Cards
Multizone Integration
Entities
Switch Platform
Multizone Enable
Enable/Disable
Display
Display
Expose
Expose
Climate Platform
Zone Entities
Sensor Platform
State Sensors
Config Entry
Configuration Flow
Setup
Read Temps
Setup
Read/Write
Control
Zone Climate Devices
Main Climate Device
Core Components
Data Update Coordinator
15s interval
Update
Execute
Execute
Automations
Safety Timer Automation
State Change
Trigger
Target Change
Temperature Change
Automation
Background Services
Safety Check Service
Update Valves Service
Calculate Main Target
Service
Use
Use
Use
Dequeue
Core Logic
Redis Client
Enqueue Jobs
Read/Write
Manage
Store/Retrieve
Store/Retrieve
Store/Retrieve
Redis Database
Global Config
Zone States
Job Queues
Locks & Status

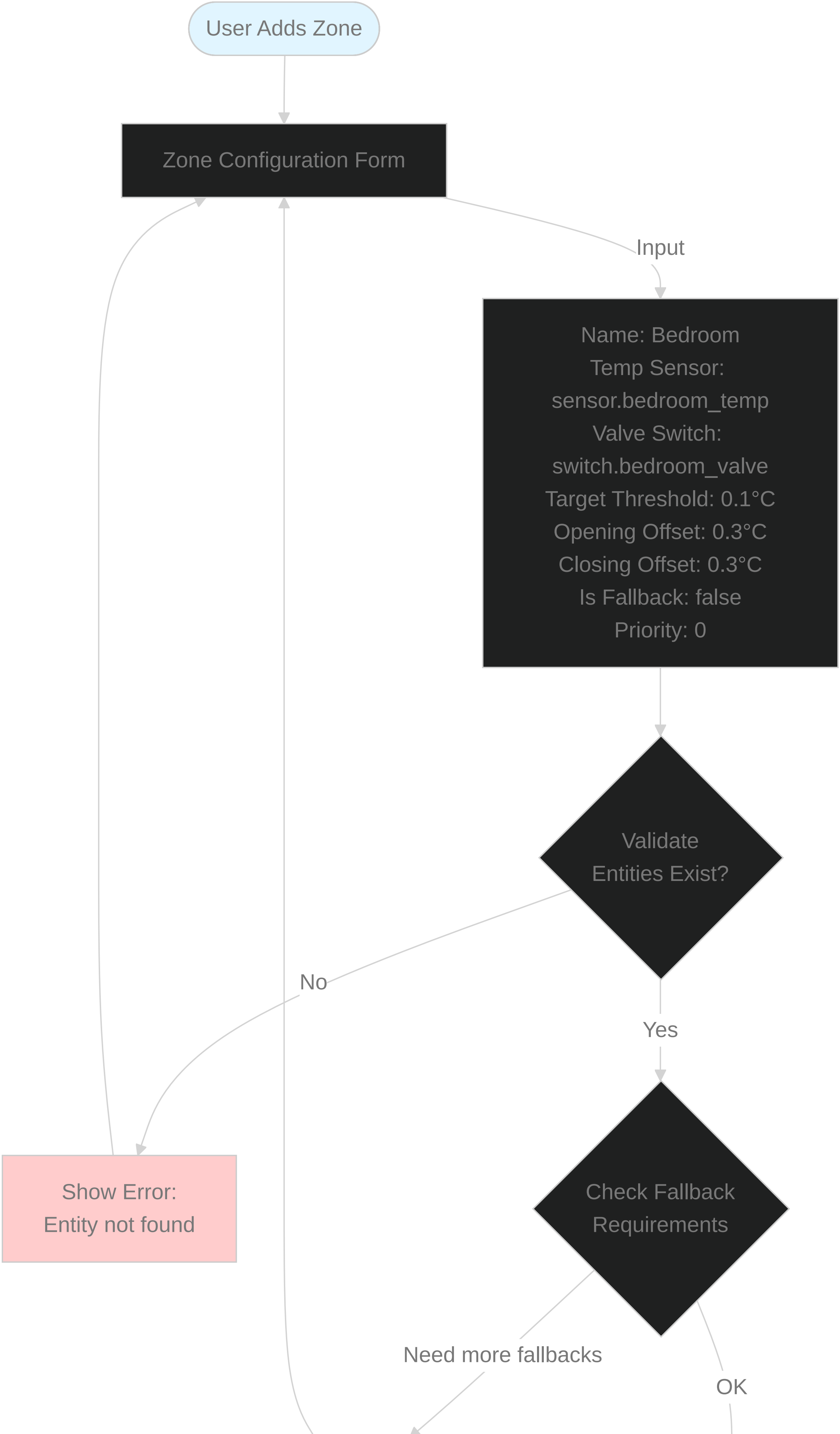

User
Sets Target
Lovelace Climate Card
Update
Zone Climate Entity
Write
Fire Event
Temperature Change Automation
Enqueue
Job Queue
Wait
Coordinator: Next Cycle
Dequeue & Execute
Calculate Main Target Job
Dequeue & Execute
Update Valves Job
Read
Read
Redis: Zone State
Calculate
New Main Target
Update
Main Climate Entity
API Call
Cloud API
Control
HVAC Unit
Determine Actions
Valve Actions
Open/Close
Valve Switches
Physical Control
Physical Valves

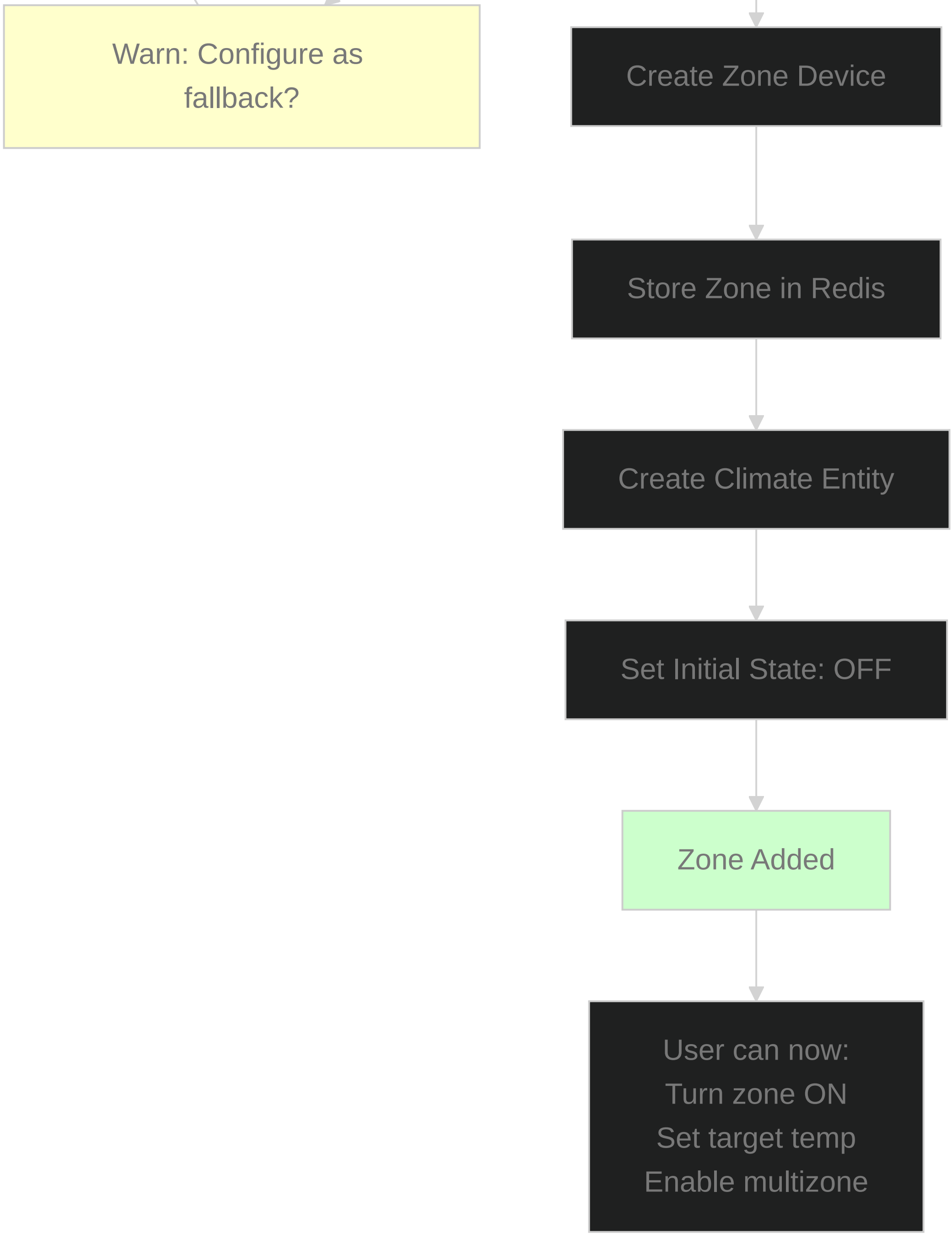

Zones with Priority
Zone A: Bedroom
Priority: 10
Target: 21°C, Current: 19°C
Deficit: 2.0°C
Sort Key: (10, 2.0)
Zone B: Kitchen
Priority: 5
Target: 22°C, Current: 19°C
Deficit: 3.0°C
Sort Key: (5, 3.0)
Zone C: Living
Priority: 0
Target: 20°C, Current: 16°C
Deficit: 4.0°C
Sort Key: (0, 4.0)
Zone D: Bathroom
Priority: 0
Target: 23°C, Current: 22°C
Deficit: 1.0°C
Sort Key: (0, 1.0)
Sorted Order (High to Low
1st: Zone A
Highest user priority
2nd: Zone B
Second user priority
3rd: Zone C
Default priority, largest deficit
4th: Zone D
Default priority, smallest deficit
Managed First
Managed Second
Managed Third
Managed Last

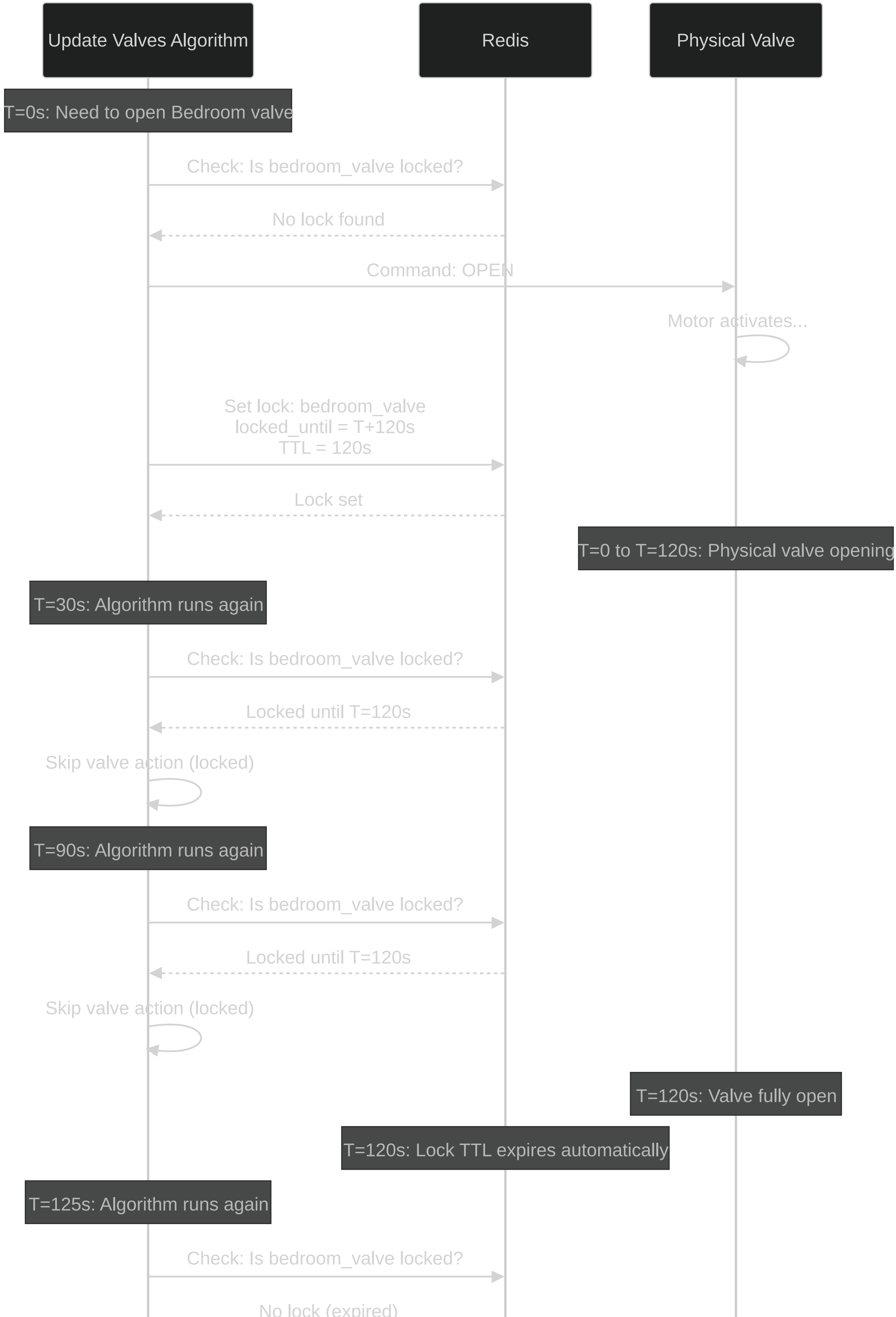

User Adds Integration
Welcome Screen
Multizone Climate
Configure Redis
Connection
Input
Host: localhost
Port: 6379
Password: optional
DB: 0
Prefix: ha_multizone
Test Redis
Connection
Failed
Success
Show Error
Select Main Climate Entity
Entity Selector
climate.main_thermostat


Configure Automation Settings
Input
Mode: Slider/Average
Slider: 50% default
Min Valves: 1
Min Temp: 18°C
Max Temp: 30°C
Change Threshold: 0.5°C
Valve Delay: 120s
Satisfaction Eps: 0.0°C
Validate Config
Invalid
Valid
Show Validation Errors
Create Config Entry
Setup Main Climate Device
Initialize Redis Schema
Setup Complete

Show Options:
Add Climate Zone

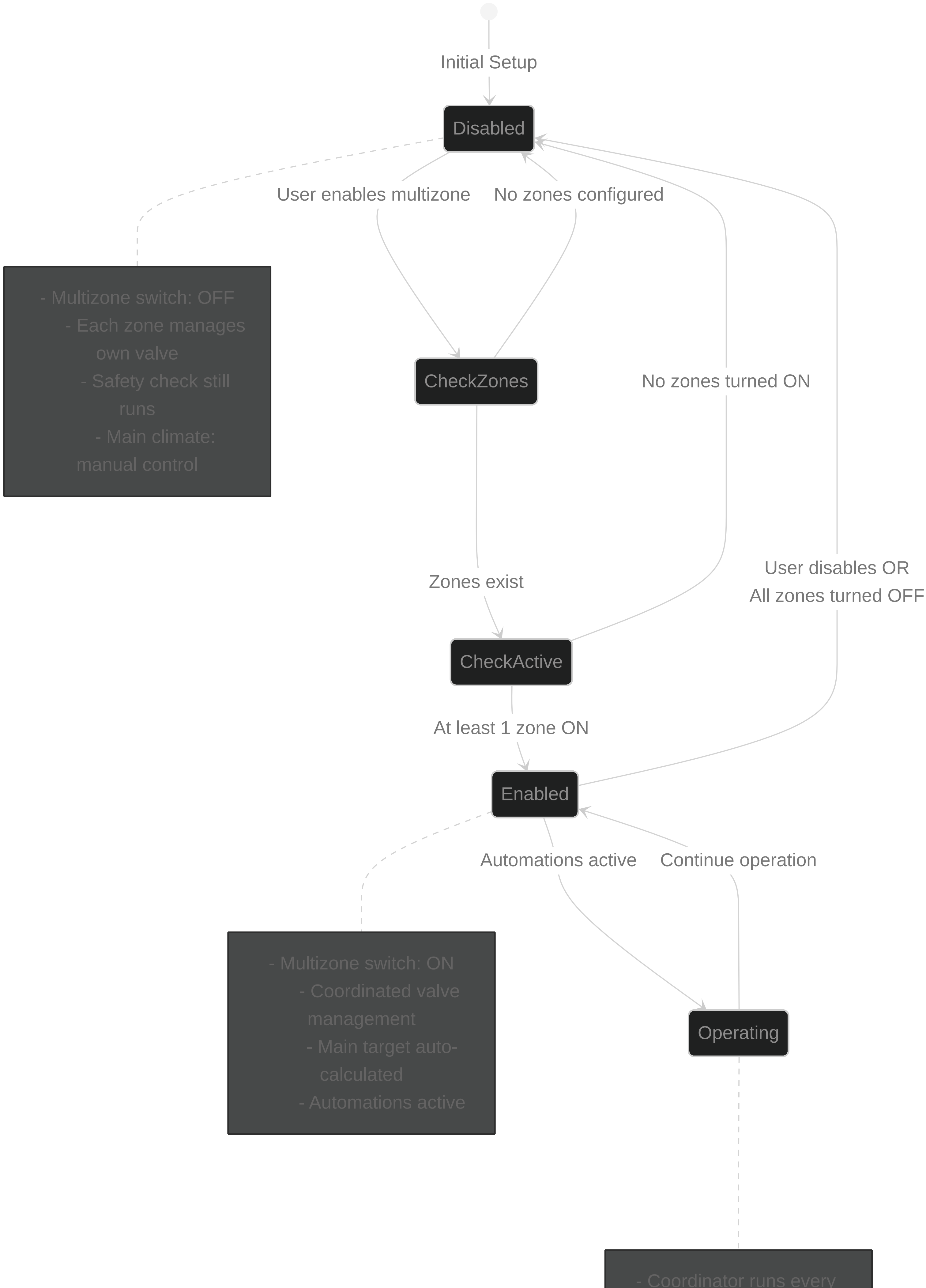

User Adds Zone
Zone Configuration Form
Input
Name: Bedroom
Temp Sensor:
sensor.bedroom_temp
Valve Switch:
switch.bedroom_valve
Target Threshold: 0.1°C
Opening Offset: 0.3°C
Closing Offset: 0.3°C
Is Fallback: false
Priority: 0
Validate
Entities Exist?
No
Show Error:
Entity not found
Yes
Check Fallback
Requirements
Need more fallbacks
OK


Warn: Configure as fallback?
Create Zone Device
Store Zone in Redis
Create Climate Entity
Set Initial State: OFF
Zone Added
User can now:
Turn zone ON
Set target temp
Enable multizone

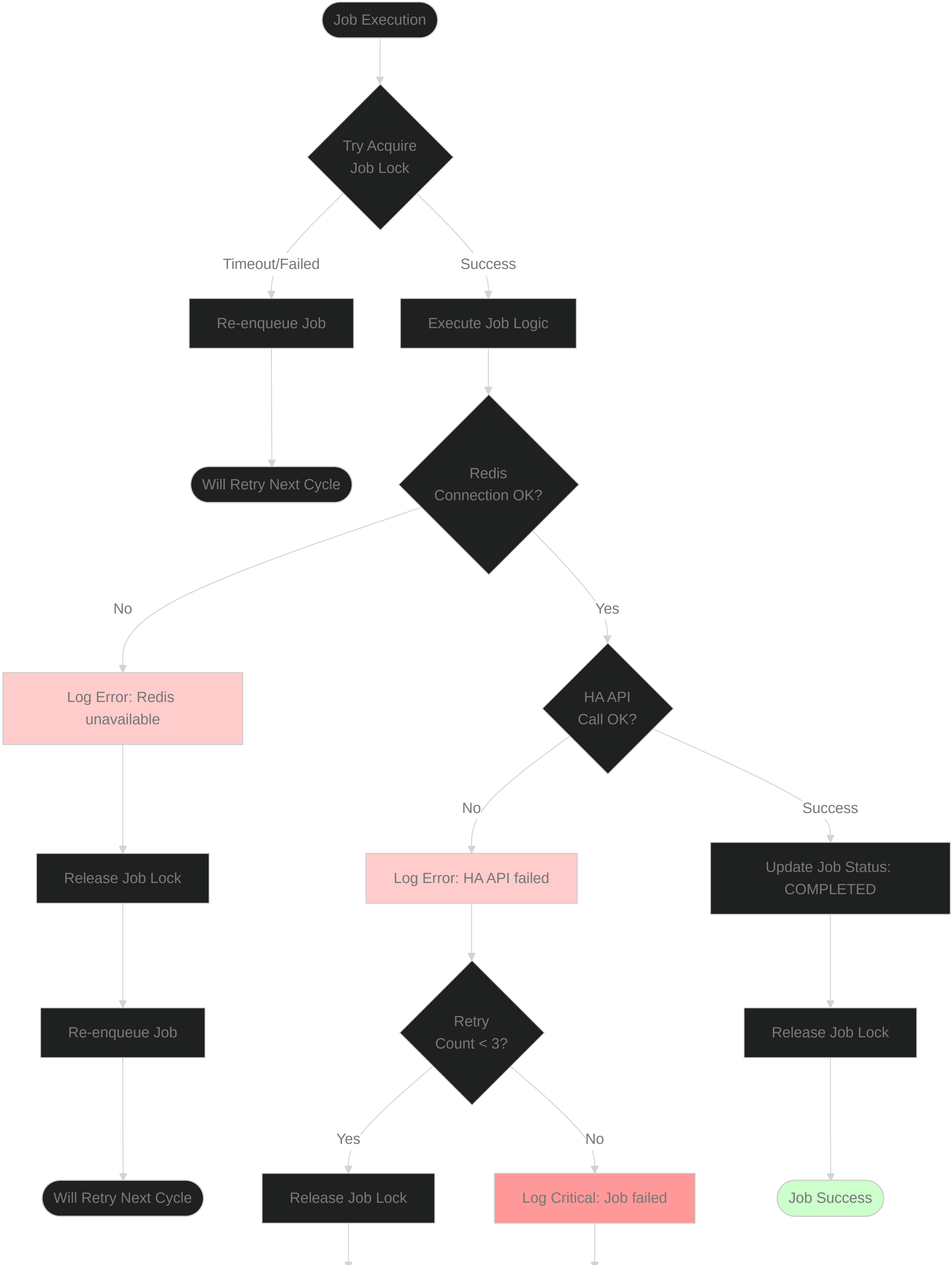


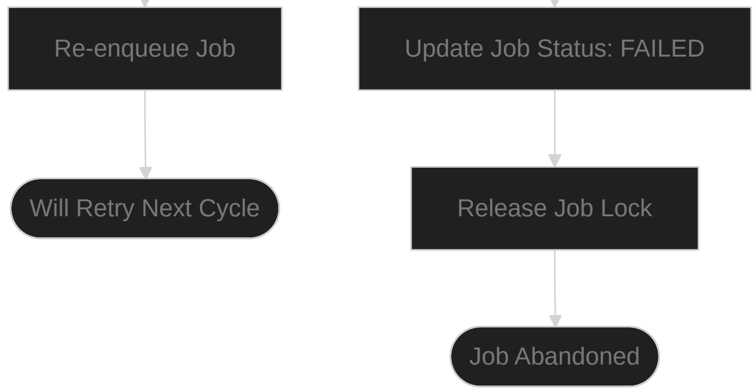
Update Valves Algorithm
Redis
Physical Valve
T=0s: Need to open Bedroom valve
Check: Is bedroom_valve locked?
No lock found
Command: OPEN
Motor activates...
Set lock: bedroom_valve
locked_until = T+120s
TTL = 120s
Lock set
T=0 to T=120s: Physical valve opening
T=30s: Algorithm runs again
Check: Is bedroom_valve locked?
Locked until T=120s
Skip valve action (locked)
T=90s: Algorithm runs again
Check: Is bedroom_valve locked?
Locked until T=120s
Skip valve action (locked)
T=120s: Valve fully open
T=120s: Lock TTL expires automatically
T=125s: Algorithm runs again
Check: Is bedroom_valve locked?
No lock (expired)


Valve can be actuated again
Update Valves Algorithm
Redis
Physical Valve


Initial Setup
Disabled
User enables multizone
No zones configured
- Multizone switch: OFF
- Each zone manages own valve
- Safety check still runs
- Main climate: manual control
CheckZones
No zones turned ON
Zones exist
User disables OR All zones turned OFF
CheckActive
At least 1 zone ON
Enabled
Automations active
Continue operation
- Multizone switch: ON
- Coordinated valve management
- Main target auto-calculated
- Automations active
Operating

15s
- Jobs process from queues
- Valves coordinated
- Main target updated


Job Execution
Try Acquire Job Lock
Timeout/Failed
Success
Re-enqueue Job
Execute Job Logic
Will Retry Next Cycle
Redis Connection OK?
No
Yes
Log Error: Redis unavailable
HA API Call OK?
No
Success
Release Job Lock
Log Error: HA API failed
Update Job Status: COMPLETED
Re-enqueue Job
Retry Count < 3?
Release Job Lock
Yes
No
Will Retry Next Cycle
Release Job Lock
Log Critical: Job failed
Job Success



Re-enqueue Job
Update Job Status: FAILED
Will Retry Next Cycle
Release Job Lock
Job Abandoned